

유니티 멀티플레이어 환경에서의 경량 암호 통신 구현



저자 1 고세화

저자 2 김재민

저자 3 이강인

지도교수 손준영

목 차

1. 서론	1
1.1. 연구 배경	1
1.2. 연구 목표	1
1.3. 연구 요구 사항 및 제약	2
2. 개발 및 테스트 환경	3
3. 배경 지식	3
3.1. TLS	3
3.2. 암호 알고리즘	4
3.3. 서버 - 클라이언트 구조	5
3.4. 게임 서버	6
3.5. 게임 클라이언트	7
4. 연구 구현	8
4.1. 블록 암호 알고리즘	8
4.1.1. AES	9
4.1.2. ARIA	12
4.1.3. HIGHT	15
4.1.4. SPECK	17
4.1.5. TWINE	19
4.2. 블록 암호 운영모드	21
4.2.1. ECB	23
4.2.2. CBC	24
4.2.3. CFB	24

4.2.4. OFB	25
4.2.5. CTR.....	26
4.2.6. GCM.....	26
4.2.7. 테스트.....	28
4.3. ECC.....	31
4.4. ECDSA.....	32
4.5. ECDH	34
4.6. HKDF	35
4.7. Handshake Protocol.....	36
4.8. 게임 서버.....	39
4.9. 게임 클라이언트	42
5. 시뮬레이션 및 적용	45
5.1. 안전한 게임 통신 환경 시뮬레이션.....	45
5.2. 블록 암호 알고리즘별 성능 측정.....	47
6. 연구 결과.....	53
6.1. 분석 및 평가.....	53
6.2. 연구의 한계 및 추가 연구 제안.....	53
7. 결론 및 향후 연구 방향.....	54
8. 멘토링 결과 반영 내용.....	55
9. 참고 문헌.....	55

1. 서론

1.1. 연구 배경

컴퓨터 산업이 발전함에 따라 컴퓨터 게임 산업 역시 급속도로 성장했다. 초기에 단순한 콘솔 게임으로 시작된 게임 산업은 하드웨어 성능의 향상과 네트워크 기술의 발전에 힘입어 다양한 장르의 고품질 게임을 제공하는 산업으로 성장했다. 특히 인터넷의 보급과 함께 멀티플레이어 게임에 대한 관심이 증가했으며, 네트워크를 통해 다른 사용자와 실시간으로 게임을 즐길 수 있게 되었다. 이러한 환경 변화는 게임 개발에 있어 네트워크 통신 기술의 중요성을 부각하였고, 유니티(Unity)와 같은 게임 엔진이 멀티플레이어 게임 개발을 위한 다양한 네트워킹 기능을 제공하는 계기가 되었다.

멀티플레이어 게임의 수요 증가로 인해 관련 보안 기술의 필요성 또한 증가했다. 멀티플레이어 게임에서는 치트 및 해킹 방지가 중요하다. 일부 플레이어가 게임 내에서 불법적인 방법으로 부당한 이익을 취하려는 시도는 게임의 공정성을 훼손하고 다른 플레이어의 경험을 저해하기 때문이다. 이러한 문제는 특히 경쟁적인 요소가 강한 FPS나 MMORPG에서 더욱 두드러진다. FPS 장르의 경우, 에임봇(aimbot, 상대 위치에 플레이어의 에임이 고정됨)이나 월해킹(wallhack, 벽 뒤의 상대 플레이어가 보임) 같은 치트 프로그램은 게임의 밸런스를 무너뜨릴 수 있다. MMORPG에서는 아이템 복제나 골드 파밍을 통해 게임 경제를 망치는 행위가 발생할 수 있다. 이외에도 게임 계정 정보 탈취나 개인 정보 유출을 노리는 피싱 공격도 증가하고 있어, 게임 보안 기술은 단순한 치트 방지를 넘어선 광범위한 보안 이슈를 다루게 되었다.

게임 보안 기술은 클라이언트 보안과 네트워크 보안으로 구분된다. 이 중 네트워크 보안은 웹 서비스의 네트워크 보안과 유사하지만, 멀티플레이어 게임은 웹 서비스보다 훨씬 낮은 네트워크 지연 속도를 유지해야 한다는 특성이 있다. 특히, FPS나 MMORPG와 같이 여러 유저가 실시간으로 상호작용을 해야 하는 경우 해당 특성이 더욱 부각된다. 하지만, 다양한 멀티플레이어 게임에서 어떤 암호 알고리즘이 구체적으로 적용되었는지는 크게 알려진 바가 없다. 따라서 본 연구에서는 멀티플레이어 게임에서 안전한 통신 환경을 구축하고, 각 블록 암호 알고리즘별 성능을 비교해 볼 것이다.

1.2. 연구 목표

본 연구에서는 유니티를 기반으로 개발된 MMORPG 게임에서 안전한 게임 통신 환경을 구현하고, 경량 암호 알고리즘을 포함한 다양한 블록 암호 알고리즘을 적용하여 각 알고리즘의 성능을 비교하려 한다. 이를 위해 일반적인 MMORPG 서버 환경과 최대한 유사하게 구현된 서버와 클라이언트를 만들고, 보안 통신을 위한 핸드셰이크(Handshake) 과정을 수행한다. 이를 통해 멀티플레이어 게임 환경에서 적합한 암호 알고리즘을 탐색하고, 유니티를 활용한 멀티플레이어 게임 개발 시 적용할 수 있는 최적의 보안 솔루션을 제시하고자 한다.

1.3. 연구 요구 사항 및 제약

네트워크 보안 통신을 위해서 최초 세션 수립 시 클라이언트가 서버를 인증한 후 공유 비밀을 생성하고, 해당 공유 비밀로부터 대칭 키(세션 키)를 유도한다. 세션 키를 통해 이후 통신에 사용되는 모든 패킷을 블록 암호 알고리즘을 통해 암호화한다. 위 과정에서 사용하는 암호 알고리즘은 다음과 같다.

- 전자 서명 알고리즘 : ECDSA(P-256, SHA-256)
- 키 교환 알고리즘 : ECDH(P-256, SHA-256)
- 키 유도 함수 : HKDF(SHA-256)
- 블록 암호 알고리즘 : AES, ARIA, HIGHT, SPECK, TWINE
- 블록 암호 운영모드 : ECB, CBC, CFB, OFB, CTR, GCM.

암호 알고리즘의 경우 라이브러리로서 이미 구현된 것도 있고, 그렇지 않은 부분도 있다. 이는 같은 암호 알고리즘이라도 구현 방식의 차이에 의해 성능이 다를 수 있다. 따라서, 해시 함수인 SHA-256을 제외한 모든 암호 알고리즘을 직접 구현하였다.

GCM은 블록 사이즈가 128 bits인 알고리즘(AES, ARIA, SPECK)에서 작동하며, 64 bits의 경우 표준 구현이 없다. GCM 구현 명세서에 블록 사이즈가 64 bits 일 때 동작하도록 고안된 알고리즘이 있지만^[1], 안전성과 IV 고유성 확보 등의 문제가 있다. 따라서, 본 연구의 범위를 벗어나기에 구현에서 제외하였다.

경량 암호 알고리즘인 HIGHT, SPECK, TWINE의 경우 리소스가 제한된 환경에서 효율적인 동작을 위해 설계되었다. 하지만, 본 연구는 일반적인 PC에서 동작하며, 이는 충분한 CPU 성능과 메모리를 제공하므로, 경량 암호 알고리즘의 설계 목적에 부합하는 리소스 절감 효과는 미미할 것이다. 그렇지만 본 연구에서 각 알고리즘의 성능은 시간을 중심으로 평가할 것이다. 처리 시간은 알고리즘이 동일한 하드웨어 환경에서 얼마나 효율적으로 작동하는지를 보여주는 중요한 지표다. 비록 PC 환경에서의 리소스 절감 효과는 크지 않더라도, 처리 속도 면에서 AES나 ARIA와 같은 전통적인 블록 암호 알고리즘과 비교했을 때 어떠한 차이를 보이는지 분석하는 것은 여전히 의미가 있다.

서버 인증의 경우 인증서의 표준 구조인 X.509가 존재하며 인증 서버가 별도로 필요하지만, 이는 연구의 범위를 벗어난다. 따라서 인증서는 메시지와 서명만 가지도록 수정하였으며, 인증 서버는 클라이언트가 코드 내에서 서버의 공개키를 직접 받도록 대체하였다. 이를 통해 구조를 간소화하면서도 인증 자체의 무결성과 신뢰성을 유지하였다.

게임 서버의 경우 MMORPG에 적합하도록 대량의 소켓 I/O를 처리할 수 있게 구현해야 한다. 이를 위해 모든 소켓 I/O는 비동기로 처리되어야 하며, 멀티스레드를 활용하는 프로그래밍이 필수적이다. 또한 용이한 테스트를 위해 유니티를 통한 클라이언트 외에도 더미 클라이언트가 필요하다. 더미 클라이언트는 단순히 서버에 패킷을 에코(echo) 하도록 지시하며 각 블록 암호 알고리즘별 성능 측정에 활용된다.

2. 개발 및 테스트 환경

- 운영체제: Windows 11 Pro 64bit
- CPU: Intel(R) Core(TM) i7-13700KF 3.40 GHz
- RAM: 32.0 GB
- GPU: NVIDIA GeForce RTX 4060 Ti
- 프로그래밍 언어: C# 12.0 (서버), C# 9.0 (클라이언트)
- 통합 개발 환경(IDE): Visual Studio 2022 Community
- 게임 엔진: Unity 2022.3.30f1 Personal

3. 배경 지식

3.1. TLS

TLS(Transport Layer Security)는 RFC에 등록된 표준[2]으로, 인터넷을 통해 전송되는 데이터를 안전하게 보호하기 위한 암호화 프로토콜이다. SSL로 알려졌던 프로토콜의 후속 버전으로, 데이터의 기밀성, 무결성, 인증을 제공하여 통신을 보호하는 역할을 한다. 핸드셰이크 과정을 통해 클라이언트와 서버는 암호화 알고리즘과 세션 키를 교환하며, 이후 통신에 사용되는 모든 데이터는 암호화되어 제 3자가 중간에서 도청하거나 변조할 수 없게 된다. TLS 1.3은 TLS 1.2에 비해 개선된 버전이다. 핸드셰이크 절차를 간소화하여 성능을 개선했으며, RSA 키 교환이나 SHA-1과 같이 더 이상 권장되지 않거나 취약점이 있는 암호화 알고리즘을 제거하였다. 아래는 TLS 1.3 핸드셰이크 과정에 대한 그림이다.



그림 1 - TLS 1.3 Handshake 과정 (출처 : [3])

위 과정을 통해 클라이언트와 서버는 공유 비밀을 생성하며, 클라이언트는 서버를 인증하게 된다. 또한, GCM이나 HMAC과 같은 추가적인 무결성 보장 메커니즘을 통해 무결성을 보장하고, 암호화된 데이터를 통해 기밀성을 확보할 수 있다.

3.2. 암호 알고리즘

블록 암호는 대칭키 암호의 일종으로, 암호화와 복호화에 동일한 암호 키를 사용하는 알고리즘이다. 평문을 고정된 크기의 블록 단위로 암호·복호화하며, 블록마다 동일한 키가 사용된다. 고정된 크기의 블록 단위로 평문을 처리하기 때문에, 마지막 블록이 고정된 크기에 맞지 않으면 패딩(padding)을 사용하여 크기를 맞춘다. 또한, 하나의 키로 여러 블록을 안전하게 처리하기 위하여 운영모드(mode of operation)라는 절차를 사용한다. 송신자와 수신자는 암호 키가 노출되지 않도록 비밀로 관리해야 하며, 송·수신자가 동일한 키를 공유해야 하므로 여러 사람과 정보 교환 시 많은 키를 관리해야 하는 어려움이 있다. 대표적인 블록 암호 알고리즘으로는 AES, ARIA가 있다.

경량 암호 알고리즘은 리소스가 제한된 환경에서도 효율적으로 동작하도록 설계된 암호 알고리즘이다. 이는 IoT 기기, 센서 네트워크, 스마트 카드와 같은 저전력, 저메모리 장치에서 사용하기 위해 최적화되었다. 경량 암호는 일반적인 블록 암호 알고리즘보다 적은 연산 자원과 메모리 공간을 사용하면서도, 일정 수준 이상의 보안성을 제공해야 한다. 따라서, 경량 암호 알고리즘은 연산의 복잡도를 줄이고 전력 소모를 최소화할 수 있도록 설계되었다. 대표적인 경량 암호 알고리즘은 HIGHT, LEA가 있다.

운영모드는 블록 암호를 사용하여 여러 블록의 데이터를 안전하게 암호화하기 위한 절차이다. 각 모드는 데이터의 기밀성과 무결성을 보장하는 데 중요한 역할을 하며, 사용 목적에 따라 다양한 운영모드가 존재한다. 대표적인 운영모드로는 ECB, CBC, CFB, OFB, CTR, GCM이 있다. GCM은 데이터의 기밀성을 제공할 뿐만 아니라, 데이터의 무결성과 인증도 제공하는 운영모드이다.

공개키 암호는 대칭키 암호와 달리 암호·복호화에 서로 다른 키(공개키와 개인키)를 사용하는 알고리즘이다. 송신자는 수신자의 공개키를 이용해 암호화하며, 수신자는 자신의 개인키로 암호화된 메시지를 복호화할 수 있다. 이는 수학적 난제를 기반으로 설계되었으며, 암호·복호화에 복잡한 수학적 연산을 사용하기 때문에 대칭키 암호에 비해 성능이 떨어질 수 있다. 하지만, 여러 송신자가 하나의 공개키로 암호화를 수행하기 때문에 사용자가 많더라도 키를 관리하는 데에 유리하다. 공개키 암호는 전자 서명과 키 교환에 사용될 수 있다.

전자 서명은 인터넷 상에서 본인임을 증명하기 위해 사용하는 방법으로, 공개키 암호를 역으로 적용하는 개념이다. 송·수신자의 역할이 반대로 되어, 개인키를 소유한 사람만이 전자 서명 알고리즘을 통해 평문에 대한 서명을 생성할 수 있다. 생성된 서명은 공개키를 통해 검증할 수 있다. 따라서, 누구나 그 서명을 검증할 수 있게 된다. 본 연구에서는 타원 곡선 기반의 전자 서명 알고리즘인 ECDSA를 사용하였다.

키 교환은 통신하는 두 당사자가 대칭키 암호에 사용될 비밀 키를 안전하게 공유하기 위한 프로토콜이다. 최초 세션 수립 후 각자 공개키/개인키 쌍을 생성하며, 자신의 공개키를 공유한 뒤 상대의 공개키와 자신의 비밀키를 통해 공유 비밀(Shared Secret)을 생성하는 것이 목적이다. 본 연구에서는 타원 곡선을 활용하는 ECDH를 사용하였다.

키 유도 함수는 임의의 길이의 입력값에서 고정된 길이의 암호 키를 생성하는 데 사용되는 함수이다. 일반적으로 공유 비밀로부터 대칭키 암호에 사용될 암호 키를 생성하는 것이 목적이다.

공유 비밀을 직접 사용하는 것은 키 길이의 문제, 하나의 공유 비밀을 여러 암호 알고리즘에서 사용하며 발생하는 문제 등의 이유로 권장되지 않는다. 본 연구에서는 HMAC 기반의 키 유도 함수인 HKDF를 사용하였다.

3.3. 서버 - 클라이언트 구조

서버 - 클라이언트 구조(Server-Client Architecture)는 네트워크 환경에서 서비스 제공자와 서비스 이용자가 상호작용하는 분산 컴퓨팅 구조를 의미한다. 이 구조는 서버가 클라이언트의 요청을 처리하고 필요한 데이터를 제공하는 방식으로 동작하며, 다양한 인터넷 기반 서비스의 기본적인 모델로 사용된다. 그림 2는 서버 - 클라이언트 구조를 나타낸다.

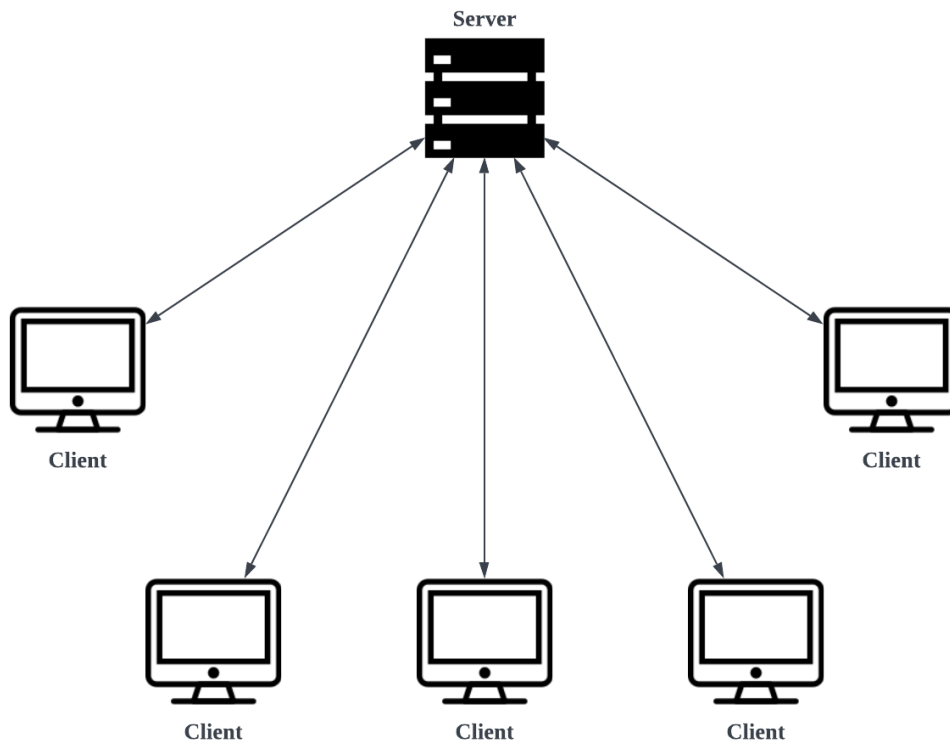


그림 2 - 서버 - 클라이언트 구조

서버는 서비스 제공자로서 각 클라이언트와 연결을 맺고, 클라이언트의 요청에 따라 데이터를 처리한 후 적절한 응답을 반환한다. 모든 데이터와 서비스를 집중시켜 관리가 용이하며, 높은 처리 능력을 요구한다. 클라이언트는 서버에 요청을 보내고 서버로부터 응답을 받아 사용한다. 일반적으로 클라이언트 간에는 직접적인 연결이 없으며, 데이터의 일관성과 무결성이 중요한 경우 대부분의 연산을 서버에서 처리하도록 한다.

3.4. 게임 서버

게임 서버는 일반적인 웹 서버와는 달리 비교적 짧은 길이의 요청을 많이 처리하며, 낮은 지연 시간이 필수적이다. 이는 게임 플레이의 경험과 직접적으로 연결되며, 다수의 유저가 실시간으로 상호작용을 할 때 이러한 특성이 더욱 두드러진다. 아래는 플레이어의 위치 정보가 상대 플레이어에게 전달되기까지를 나타내는 그림이다.

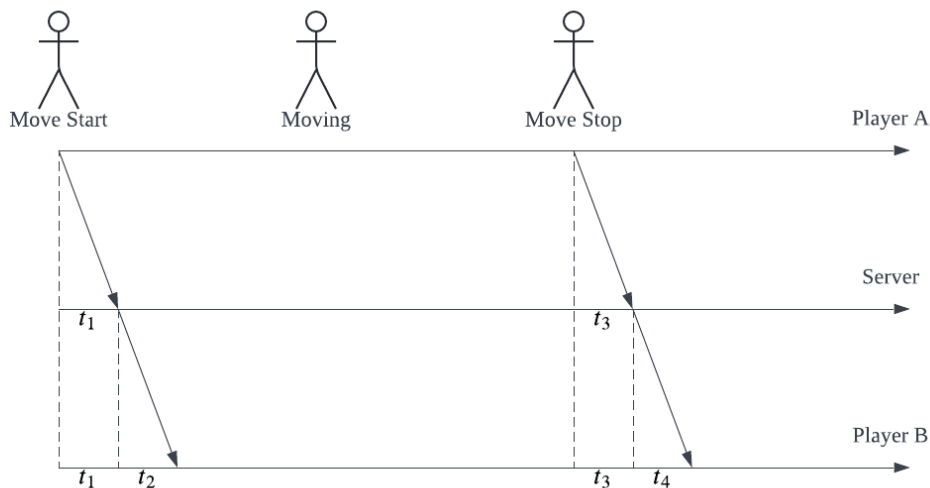


그림 3 - 플레이어의 위치 정보를 담은 패킷을 송신하는 과정

각 플레이어의 상태를 매 프레임마다 송신하여 업데이트하는 것이 가장 이상적이지만, 트래픽과 데이터 처리 문제로 인해 현실적으로 어렵다. 따라서, 특정 이벤트가 발생할 때만 해당 이벤트를 시뮬레이션 하는 방식으로 진행한다. 위 그림에서 Player A가 이동을 시작하면 Server로 자신의 위치 정보를 담은 패킷을 송신하고, Server는 이를 검증한 뒤 Player B에게 전달한다. Player B는 해당 패킷을 수신한 뒤 Player A의 위치 정보를 업데이트 한다. 이때, Player B는 Player A가 패킷을 송신한 시점으로부터 $t_1 + t_2$ 만큼 지연된 후 패킷을 수신하게 된다. 마찬가지로, Player A가 이동을 멈출 때도 $t_3 + t_4$ 만큼의 지연이 발생한 후 패킷이 전달된다. 이러한 지연으로 인해 시뮬레이션 간 시간 차이가 발생하며, 지연시간이 적을수록 정확한 동기화가 가능하다.

게임 서버는 공정한 게임 진행을 위해 클라이언트 요청을 추가로 검증하며, 이로 인해 높은 처리 능력이 요구된다. 따라서, 멀티스레드를 활용한 프로그래밍이 필수적이다. .NET Framework는 소켓 I/O 처리에 뛰어난 성능을 제공하는 고수준의 API를 제공한다. 본 연구에서는 IOCP 서버와 비동기 I/O 함수를 활용해 고성능의 게임 서버를 작성하였다.

IOCP(IO Completion Port)는 Windows 운영체제에서 제공하는 비동기 I/O 작업을 효율적으로 처리하기 위해 제공되는 고성능 I/O 모델이다. 비동기 I/O 작업은 운영체제가 관리하며, 작업이 완료되면 해당 작업의 정보가 Completion Port에 전달된다. 스레드는 운영체제에 의해 적절히 관리되며, 스레드의 개수와 처리 방식 또한 운영체제가 책임진다. 따라서 프로그래머는 IOCP 서버를 작성할 때 스레드 관리나 비동기 I/O를 직접 처리할 필요가 없으며, 이는 매우 편리하고

안전할 뿐만 아니라 성능도 보장된다.

세션은 클라이언트와 서버 간의 상호작용을 하나의 흐름으로 관리하는 논리적인 단위로, 클라이언트의 여러 요청과 서버의 응답을 연결하는 역할을 한다. 웹 서버의 경우, 클라이언트와 서버 간의 연결을 지속적으로 유지하지 않고, 각 클라이언트에게 고유한 세션 ID를 발급하여 이를 통해 상태를 관리한다. 클라이언트가 서버에 요청을 보낼 때 이 세션 ID를 함께 전송하면, 서버는 이를 이용해 해당 클라이언트를 식별하고 이전의 상태 정보를 유지할 수 있다. 반면, 게임 서버는 웹 서버와 달리 TCP 연결을 지속적으로 유지하며, 클라이언트는 연결된 소켓을 통해 직접 I/O 작업을 수행하여 서버와 실시간으로 상호작용한다.

3.5. 게임 클라이언트

게임 클라이언트는 서버와 실시간으로 데이터를 주고받으며, 이를 바탕으로 게임 상태를 클라이언트 화면에 반영하는 역할을 한다. 플레이어의 움직임, 상호작용, 게임 상태 변화 등 핵심적인 정보는 서버와 클라이언트 간의 지속적인 데이터 교환을 통해 처리된다. 본 연구에서는 유니티 게임 엔진을 사용하여 게임 클라이언트를 구현하였다. 유니티를 통한 멀티플레이어 게임 개발 시 주의해야 할 두 가지 핵심 사항이 있다.

첫 번째로, `MonoBehaviour` 클래스이다. `MonoBehaviour`는 유니티에서 게임 오브젝트(`GameObject`)에 동작을 부여하는 스크립트의 기본 클래스이다. 유니티에서 작성되는 모든 스크립트는 `MonoBehaviour`를 상속받아야 하며, 이를 통해 게임 오브젝트의 행동을 제어하고 게임 로직을 구현할 수 있다. `MonoBehaviour`는 유니티의 게임 루프와 밀접하게 연동되며, 여러 이벤트 기반 메서드를 제공한다. 주요 메서드는 다음과 같다.

- `Start()` : 게임 오브젝트가 활성화될 때 한 번 호출되며, 주로 초기화 작업에 사용된다. 예를 들어, 초기 상태 설정이나 필수 자원의 로딩 작업이 여기서 이루어진다.
- `Update()` : 매 프레임마다 호출되며, 게임 오브젝트의 상태를 지속적으로 갱신하는데 사용된다. 주로 사용자 입력 처리, 오브젝트의 움직임, 게임 상태 업데이트 등이 이곳에서 처리된다.

이외에도 `FixedUpdate()`, `LateUpdate()`, `OnTriggerEnter()` 등의 다양한 메서드를 통해 물리 연산, 충돌 처리, 비동기 작업 등을 구현할 수 있다. 이처럼 `MonoBehaviour`는 게임 오브젝트의 행동을 제어하는 핵심적인 역할을 한다.

두 번째로, `GameObject` 접근의 스레드 제한이다. `GameObject`는 모든 게임 요소의 기본 단위로, 캐릭터, 환경, UI 등 모든 시각적 요소와 상호작용 가능한 오브젝트는 `GameObject`로 표현된다. 유니티는 기본적으로 싱글 스레드로 작동하며, `GameObject`와 관련된 모든 작업은 메인 스레드에서 처리된다. 이는 유니티 엔진이 게임 루프와 렌더링을 메인 스레드에서 관리하는 특성 때문이다. 따라서 서버와의 비동기 통신으로 데이터를 수신한 후, 해당 데이터를 바탕으로 `GameObject`를 수정하거나 상태를 업데이트하려면 반드시 메인 스레드에서 작업이 이루어져야 한다. 만약 다른 스레드에서 `GameObject`에 접근하거나 이를 조작하려고 시도하면, 유니티는 예

외를 발생시키며 클라이언트가 비정상적으로 종료될 수 있다.

4. 연구 구현

4.1. 블록 암호 알고리즘

본 연구에서 다룰 블록 암호 알고리즘의 경우 실험의 공정성을 위해 키 길이를 128 bits로 제한하였으며, 블록 사이즈 또한 128 bits 혹은 64 bits로 제한하였다. 또한 일관된 인터페이스를 제공하기 위해 다음과 같은 인터페이스를 구현하는 식으로 진행하였다.

```
public interface IEncryptionAlgorithm
{
    byte[] Encrypt(byte[] plainText);
    byte[] Decrypt(byte[] cipherText);
    string AlgorithmName { get; }
    int GetBlockSize();
}

public abstract class EncryptionAlgorithm : IEncryptionAlgorithm
{
    protected EncryptionAlgorithm(byte[] key)
    {
        if (key.Length != 16)
            throw new ArgumentException("Key must be exactly 16 bytes long.");

        this.key = new byte[16];
        Array.Copy(key, this.key, 16);
    }
    public abstract byte[] Encrypt(byte[] plainText);
    public abstract byte[] Decrypt(byte[] cipherText);
    public abstract string AlgorithmName { get; }
    public abstract int GetBlockSize();

    protected readonly byte[] key;
}
```

모든 블록 암호 알고리즘은 위 인터페이스를 상속받아 구현하며, 이를 통해 운영모드나 다른 클래스가 활용할 때 일관된 인터페이스를 제공할 수 있다. 그림 4는 블록 암호 알고리즘의 클래스 다이어그램이다.

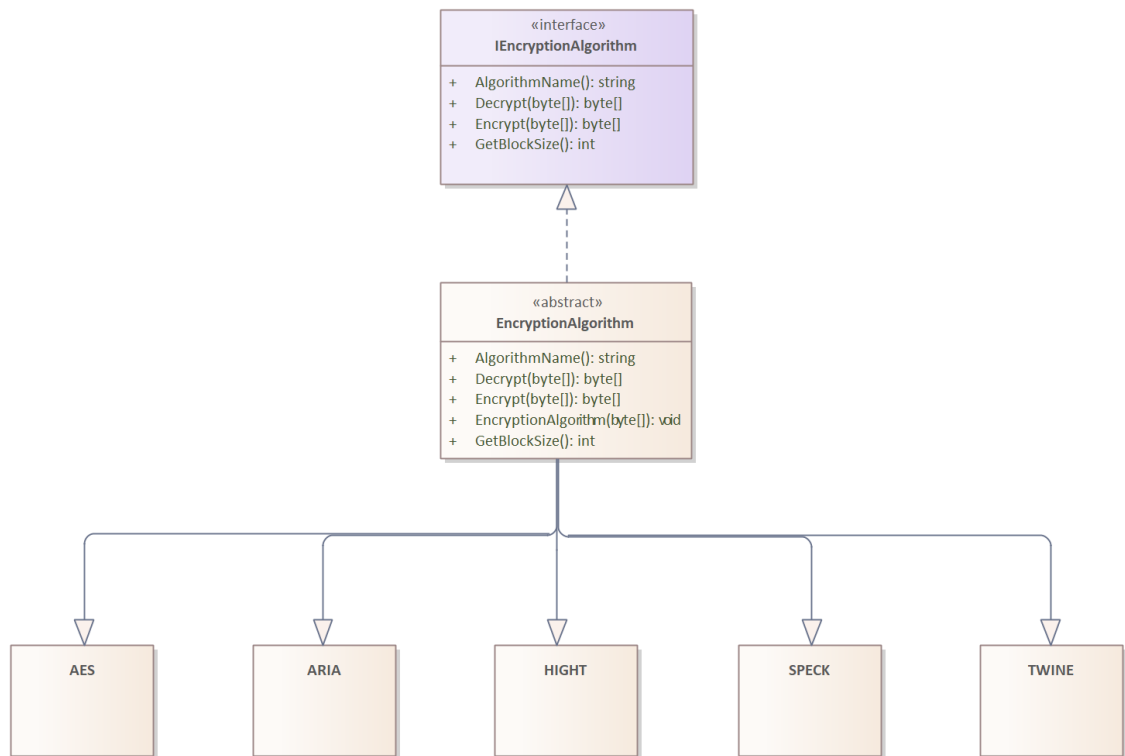


그림 4 - 블록 암호 알고리즘 클래스 다이어그램

4.1.1. AES

AES는 NIST에서 제정한 표준[4]이며, 가장 대중적으로 사용되는 블록 암호 알고리즘이다. 블록 사이즈는 128 bits, 키 길이는 128, 192, 256 bits가 가능하며, 본 연구에서는 AES-128을 사용하였다. AES-128은 총 10개의 라운드로 구성되어 있으며, 각 라운드에서 사용될 라운드 키는 초기 키로부터 미리 계산된다. 해당 과정을 Key Schedule(또는 Key Expansion)이라 하며, 이 과정에서 생성된 라운드 키들이 각 라운드에서 사용된다. 그림 5는 Key Schedule 과정을 나타낸다.

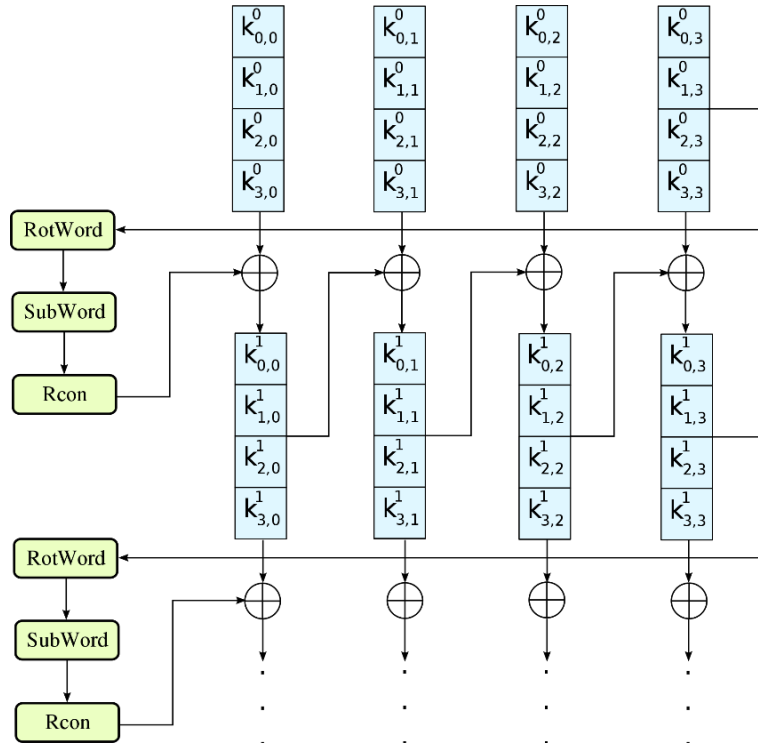


그림 5 - AES의 Key Schedule (출처 : [5])

이를 통해 각 라운드에서 사용할 10개의 라운드 키가 계산된다. 위 과정을 수식으로 표현하면 다음과 같다.

$$\text{RotWord}([b_0 \ b_1 \ b_2 \ b_3]) = [b_1 \ b_2 \ b_3 \ b_0]$$

$$\text{SubWord}([b_0 \ b_1 \ b_2 \ b_3]) = [S(b_0) \ S(b_1) \ S(b_2) \ S(b_3)]$$

$i = 0 \dots 4R - 1$:

$$W_i = \begin{cases} K_i & \text{if } i < N \\ W_{i-N} \oplus \text{SubWord}(\text{RotWord}(W_{i-1})) \oplus rcon_{i/N} & \text{if } i \geq N \text{ and } i \equiv 0 \pmod{N} \\ W_{i-N} \oplus \text{SubWord}(W_{i-1}) & \text{if } i \geq N, N > 6, \text{ and } i \equiv 4 \pmod{N} \\ W_{i-N} \oplus W_{i-1} & \text{otherwise.} \end{cases}$$

$K_0, K_1, K_2, \dots, K_{N-1}$ 은 32-bit words 단위로 본 초기 키이다. N 은 32-bit words 단위로 본 키의 길이이다. $rcon$ 은 각 라운드별 Key Schedule에 필요한 상수이다. 위 과정을 통해 각 라운드에 필요한 라운드 키를 계산한다.

위 과정을 거쳐 각 라운드에 사용할 라운드 키가 계산되었다면, 암호화와 복호화에 필요한 준비가 끝난다. 그림 6은 AES의 암호 · 복호화 과정을 나타낸다.

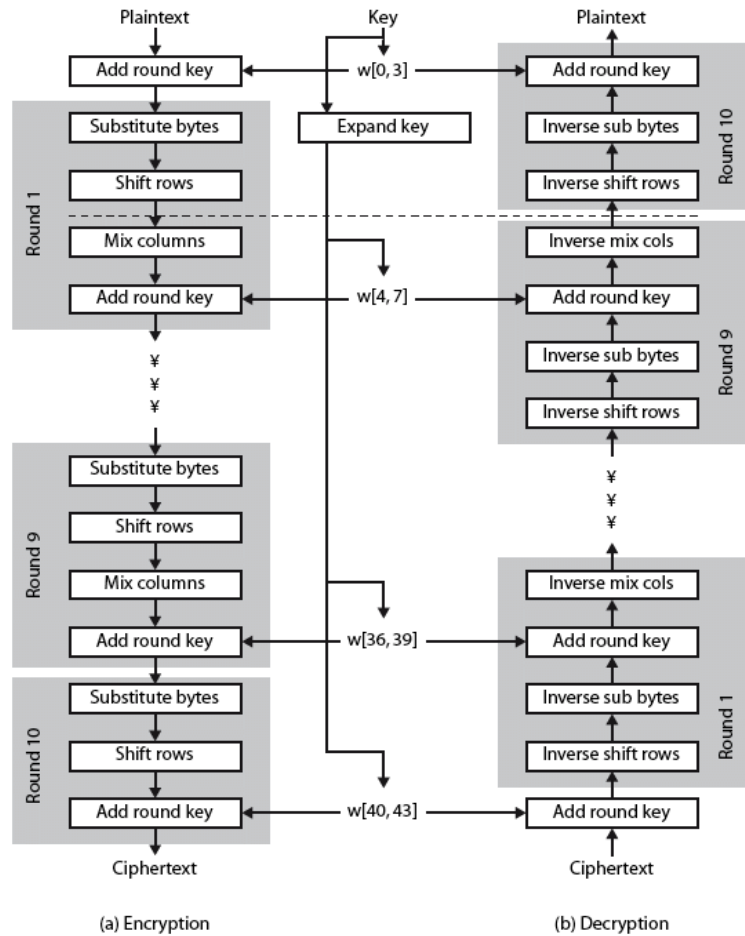


그림 6 - AES의 암호화 및 복호화 과정 (출처 : [4])

암호화와 복호화는 서로 역순 관계이며, 암호화 과정의 단계별 설명은 다음과 같다.

- Add round key : State에 라운드 키를 XOR 하는 단계이다.
- Substitute bytes : State의 각 Byte를 SBox를 통해 치환하는 단계이다.
- Shift rows : State의 각 Row를 일정 칸만큼 Left Rotate 하는 단계이다.
- Mix columns : State의 각 Column을 정해진 행렬과 곱하는 단계이다.

위 과정을 통해 평문이 암호화되며, 복호화 또한 암호화 과정을 역으로 진행한다.

AES의 구현은 NIST 표준을 참고해 구현하였으며, 테스트 결과는 다음과 같다.

```

----- AES-128 Test -----

key:
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

plainText:
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

cipherText:
C6 A1 3B 37 87 8F 5B 82 6F 4F 81 62 A1 C8 D8 79

decryptedText:
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

```

그림 7 - AES 테스트 결과, 암호화 및 복호화가 정상적으로 이루어짐을 보여준다.

4.1.2. ARIA

ARIA는 KS에서 제정한 표준 블록 암호 알고리즘[6]이다. 블록 사이즈는 128 bits, 키 길이는 128, 192, 256 bits가 가능하며, 본 연구에서는 ARIA-128을 사용하였다. ARIA-128은 총 12개의 라운드로 구성되어 있으며, AES와 유사하게 Key Schedule이 필요하다. 하지만, 암호화와 복호화에 사용하는 라운드 키가 다르며, 그 계산 방식도 차이가 있다.

ARIA의 Key Schedule 과정은 크게 초기화 과정과 라운드 키 생성 과정으로 나뉜다. 먼저, 초기화 과정에서는 암호 · 복호화에서 한 라운드로 사용되는 F 함수를 기반으로 256 bits 입출력의 3라운드 Feistel 암호를 이용하여, 암호 키(MK)로부터 네 개의 128 bits 값인 W_0, W_1, W_2, W_3 를 생성한다. 본 연구에서는 암호 키 길이가 128 bits로 고정되었기 때문에, Feistel 암호의 입력으로 필요한 256 bits는 암호 키를 제외한 나머지 128bits를 0으로 패딩하는 것으로 구성한다.

$$KL \parallel KR = MK \parallel 0 \dots 0$$

F_0, F_e 를 각각 홀수 라운드 함수, 짝수 라운드 함수라고 할 때 W_0, W_1, W_2, W_3 의 생성 과정은 다음과 같다.

$$\begin{aligned}
 W_0 &= KL & W_1 &= F_0(W_0, CK_1) \oplus KR \\
 W_2 &= F_e(W_1, CK_2) \oplus W_0 & W_3 &= F_0(W_2, CK_3) \oplus W_1
 \end{aligned}$$

CK_i 는 Key Schedule 과정에서만 사용하는 상수이다. 이후 아래 과정을 거쳐 각 라운드에 사용할 라운드 키를 생성한다. $W_i^{\ggg n}$ 은 W_i 를 n 칸 만큼 Right Rotate 한다는 의미이다.

$$\begin{aligned}
ek_1 &= (W_0) \oplus (W_1 \ggg^{19}), & ek_2 &= (W_1) \oplus (W_2 \ggg^{19}), \\
ek_3 &= (W_2) \oplus (W_3 \ggg^{19}), & ek_4 &= (W_0 \ggg^{19}) \oplus (W_3), \\
ek_5 &= (W_0) \oplus (W_1 \ggg^{31}), & ek_6 &= (W_1) \oplus (W_2 \ggg^{31}), \\
ek_7 &= (W_2) \oplus (W_3 \ggg^{31}), & ek_8 &= (W_0 \ggg^{31}) \oplus (W_3), \\
ek_9 &= (W_0) \oplus (W_1 \lll^{61}), & ek_{10} &= (W_1) \oplus (W_2 \lll^{61}), \\
ek_{11} &= (W_2) \oplus (W_3 \lll^{61}), & ek_{12} &= (W_0 \lll^{61}) \oplus (W_3), \\
ek_{13} &= (W_0) \oplus (W_1 \lll^{31}), & ek_{14} &= (W_1) \oplus (W_2 \lll^{31}), \\
ek_{15} &= (W_2) \oplus (W_3 \lll^{31}), & ek_{16} &= (W_0 \lll^{31}) \oplus (W_3), \\
ek_{17} &= (W_0) \oplus (W_1 \lll^{19})
\end{aligned}$$

복호화 과정에서 사용할 라운드 키의 Key Schedule은 암호화에서 사용한 라운드 키로부터 유도되며, 확산 함수 A의 출력이 복호화 라운드 키가 된다. 라운드 수가 n일 때, 복호화 라운드 키 dk_i 는 다음과 같다.

$$dk_1 = ek_{n+1}, dk_2 = A(ek_n), dk_3 = A(ek_{n-1}), \dots, dk_n = A(ek_2), dk_{n+1} = ek_1.$$

위 과정을 거쳐 각 라운드에 사용할 라운드 키가 계산되었다면, 암호화와 복호화에 필요한 준비가 끝난다. 그림 8은 ARIA의 암호·복호화 과정을 나타낸다.

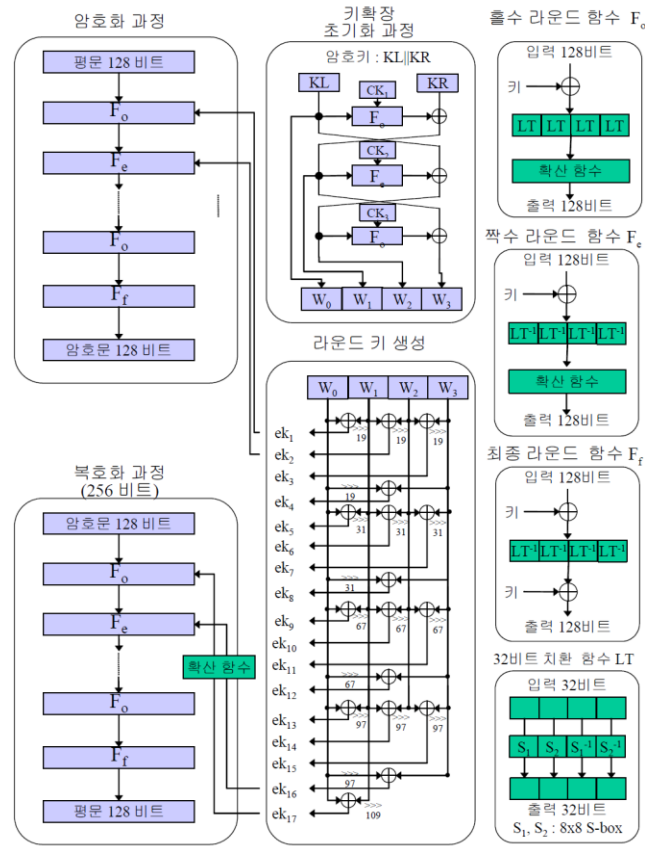


그림 8 - ARIA의 암호화 및 복호화 과정 (출처 : [6])

위 과정은 크게 세 부분으로 나뉜다.

- AddRoundKey : 128 bits의 라운드 키를 입력 128 bits와 XOR 한다.
- Substitution Layer : 두 유형의 치환 계층이 있으며 2종의 8 bits 입출력 S-box와 그 역으로 구성된다. 유형 1이 홀수 라운드에 사용되며, 유형 2가 짝수 라운드에 사용된다. 그림 9는 각 유형별 Substitution Layer이다.
- Diffusion Layer : 간단한 16 x 16 involution 이진 행렬을 사용한 바이트 간의 확산 함수로 구성되어 있다. 그림 10은 각 유형별 Diffusion Layer이다.

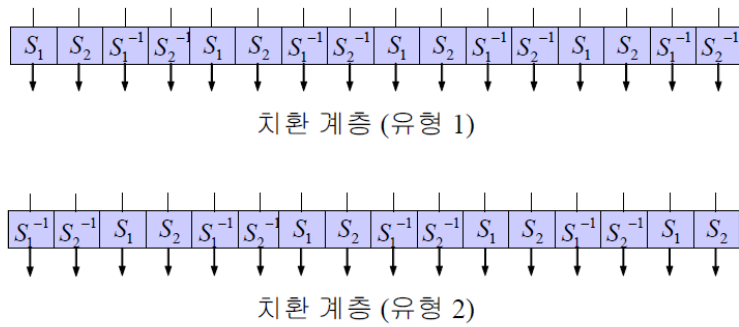


그림 9 - ARIA의 Substitution Layer (출처 : [6])

$$\begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \\ y_8 \\ y_9 \\ y_{10} \\ y_{11} \\ y_{12} \\ y_{13} \\ y_{14} \\ y_{15} \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \\ x_9 \\ x_{10} \\ x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \\ x_{15} \end{pmatrix}$$

그림 10 - ARIA의 Diffusion Layer (출처 : [6])

KISA에서 제시한 ARIA 구현 명세서와 소스 코드를 참고해 구현하였으며, 테스트 결과는 다음과 같다.

```

----- ARIA-128 Test -----

key:
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

plainText:
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

cipherText:
FA 28 27 D1 43 6C 8A 81 99 73 43 6E 60 AC 47 90

decryptedText:
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

```

그림 11 - ARIA 테스트 결과, 암호화 및 복호화가 정상적으로 이루어짐을 보여준다.

4.1.3. HIGHT

HIGHT는 RFID, USN 등과 같이 저전력, 경량화를 요구하는 컴퓨팅 환경에서 사용하기 위해 KISA와 고려대가 공동으로 개발한 블록 암호 알고리즘[7]이다. 블록 사이즈는 64 bits, 키 길이는 128 bits이다. 8 bits 단위의 기본적인 산술 연산들만으로 이루어졌으며, 앞서 본 AES, ARIA에 비해 간단한 알고리즘 구조로 설계되었다. HIGHT 또한 Key Schedule 과정을 통해 라운드 키를 계산해야 하며, 아래는 Key Schedule 과정을 수식으로 나타낸 것이다.

$$WK_i = \begin{cases} MK_{i+12} & , 0 \leq i \leq 3 \\ MK_{i-4} & , 4 \leq i \leq 7 \end{cases}$$

```

For i=0 to 7
  For j=0 to 7
    SK16·i+j ← MKj-i mod 8 ⊕ δ16·i+j;
  For j=0 to 7
    SK16·i+j+8 ← MK(j-i mod 8)+8 ⊕ δ16·i+j+8;

```

MK는 HIGHT의 초기 키이다. WK와 SK는 각각 화이트닝키, 서브키이며 라운드 키로 사용된다. 암호화와 복호화에 같은 라운드 키를 사용하며, 적용 순서만 달라진다. δ_i는 LFSR(Left Feedback Shift Register)이며, 다음과 같이 계산된다.

$$\delta_0 = (s_6, s_5, s_4, s_3, s_2, s_1, s_0) = (1, 0, 1, 1, 0, 1, 0)_2$$

$$s_{i+6} = s_{i+2} \oplus s_{i-1}, \quad \delta_i = (s_{i+6}, s_{i+5}, s_{i+4}, s_{i+3}, s_{i+2}, s_{i+1}, s_i), \quad 1 \leq i \leq 127$$

HIGHT의 암호화 라운드 총 32 라운드이며 첫 번째 라운드 함수의 입력인 평문 P는 다음과 같이 Byte(8 bits) 단위로 표현된다.

$$X_0 = X_{0,7} \parallel X_{0,6} \parallel \cdots \parallel X_{0,0}$$

X_{i,j}에서 i는 라운드의 번호, j는 라운드 입력 Byte의 순서를 나타낸다. 이를 토대로 아래 과정을 통해 평문을 암호화한다.

$$\begin{aligned}
X_{0,i} &= P_i, \quad i = 1, 3, 5, 7 \\
X_{0,0} &= P_0 \boxplus WK_0 \\
X_{0,2} &= P_2 \oplus WK_1 \\
X_{0,4} &= P_4 \boxplus WK_2 \\
X_{0,6} &= P_6 \oplus WK_3 \\
X_{i,j} &= X_{i-1,j-1}, \quad j = 1, 3, 5, 7 \\
X_{i,0} &= X_{i-1,7} \oplus (F_0(X_{i-1,6}) \boxplus SK_{4i-1}) \\
X_{i,2} &= X_{i-1,1} \boxplus (F_0(X_{i-1,0}) \oplus SK_{4i-4}) \\
X_{i,4} &= X_{i-1,3} \oplus (F_0(X_{i-1,2}) \boxplus SK_{4i-3}) \\
X_{i,6} &= X_{i-1,5} \boxplus (F_0(X_{i-1,4}) \oplus SK_{4i-2}) \\
X_{32,i} &= X_{31,i}, \quad i = 0, 2, 4, 6 \\
X_{32,1} &= X_{31,1} \boxplus (F_1(X_{31,0}) \oplus SK_{124}) \\
X_{32,3} &= X_{31,3} \oplus (F_0(X_{31,2}) \boxplus SK_{125}) \\
X_{32,5} &= X_{31,5} \boxplus (F_1(X_{31,4}) \oplus SK_{126}) \\
X_{32,7} &= X_{31,7} \oplus (F_0(X_{31,6}) \boxplus SK_{127})
\end{aligned}$$

위 과정에서 사용되는 보조 함수 F_0, F_1 은 다음과 같다.

$$\begin{aligned}
F_0(X) &= X^{\ll 1} \oplus X^{\ll 2} \oplus X^{\ll 7} \\
F_1(X) &= X^{\ll 3} \oplus X^{\ll 4} \oplus X^{\ll 6}
\end{aligned}$$

그림 12는 HIGHT의 암호화 과정을 그림으로 나타낸 것이다.

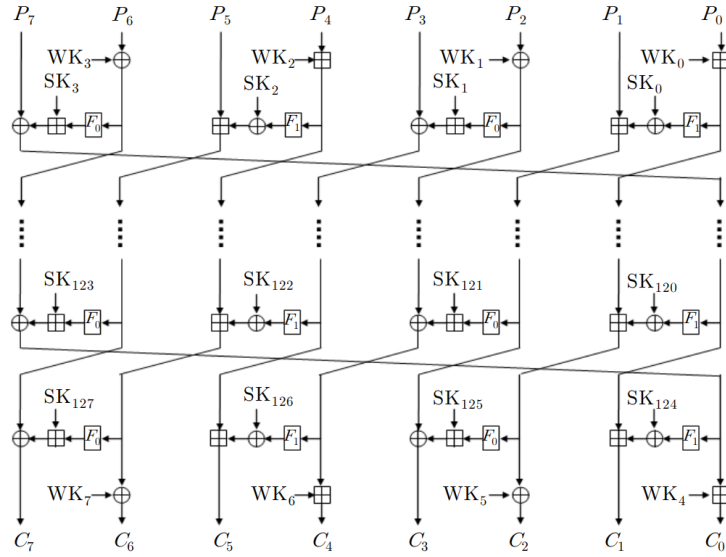


그림 12 - HIGHT의 암호화 과정 (출처 : [7])

복호화 과정은 암호화 과정을 역으로 진행하면 된다. 그림 13은 HIGHT의 암호·복호화 과정 전체를 나타낸다.

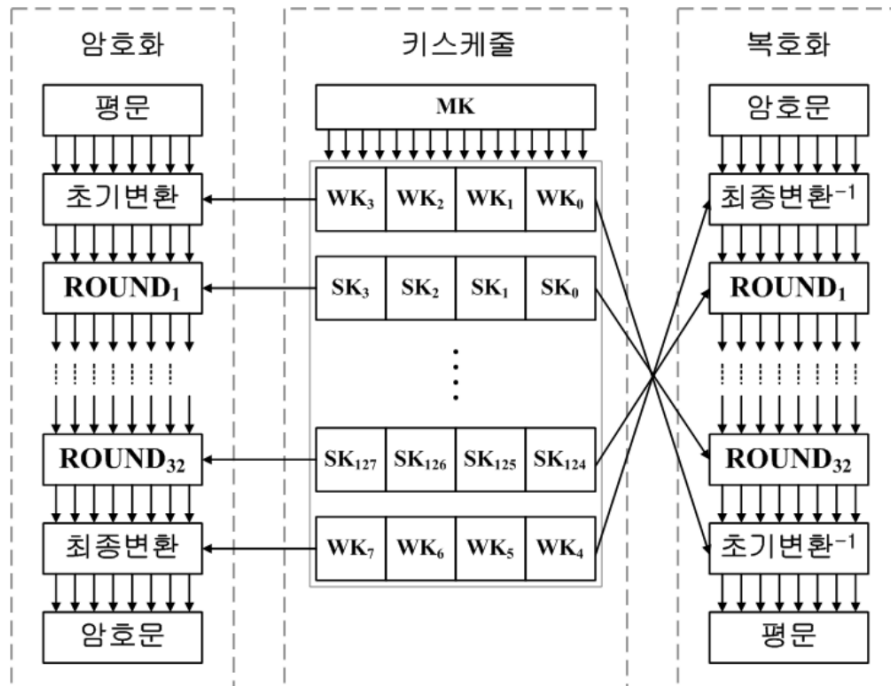


그림 13 - HIGHT의 암호화 및 복호화 과정 (출처 : [7])

KISA에서 제시한 HIGHT 구현 명세서와 소스 코드를 참고해 구현하였으며, 테스트 결과는 다음과 같다.

```

----- HIGHT-128 Test -----

key:
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

plainText:
00 00 00 00 00 00 00 00

cipherText:
5A 63 23 3E 5E F8 BF C1

decryptedText:
00 00 00 00 00 00 00 00

```

그림 14 - HIGHT 테스트 결과, 암호화 및 복호화가 정상적으로 이루어짐을 보여준다.

4.1.4. SPECK

SPECK은 NSA에서 제시한 경량 암호 알고리즘[8]으로, 소프트웨어 구현에 최적화되어 있다. 다른 경량 암호 알고리즘과 마찬가지로, 리소스가 상당히 제약된 장치에서 동작하기 위해 고안되었다. SPECK은 앞서 본 어떤 블록 암호 알고리즘보다도 구현이 간단하다. 사용 가능한 블록 사이즈와 키 길이는 다음 표와 같다.

block size	key sizes
32	64
48	72, 96
64	96, 128
96	96, 144
128	128, 192, 256

표 1 - SPECK에서 사용 가능한 블록 사이즈와 키 길이

본 연구에서는 블록 사이즈와 키 길이로 각각 128, 128 bits를 선택하였다. 앞서 본 블록 암호 알고리즘처럼 SPECK 또한 Key Schedule 과정을 통해 라운드 키를 생성해야 한다. 블록 사이즈가 128 bits일 경우 1 word를 64 bits로 보며, 암호화와 복호화에 총 32 라운드가 필요하다.

라운드 수를 T , 초기 키를 K 라고 하자, $K = (\ell_{m-2}, \dots, \ell_0, k_0)$ where $\ell_i, k_0 \in GF(2)^n$ 이라 표현할 수 있다. 이때 $0 \leq i < T$, $\min\{2, 3, 4\}$ 에 대해 i^{th} 라운드 키 k_i 는 다음과 같다.

$$\begin{aligned}\ell_{i+m-1} &= (k_i + S^{-\alpha}\ell_i) \oplus i \text{ and} \\ k_{i+1} &= S^{\beta}k_i \oplus \ell_{i+m-1}.\end{aligned}$$

S^j, S^{-j} 는 각각 Left Rotate, Right Rotate를 의미한다.

SPECK의 암호화는 128 bits의 평문을 1 word 단위로 분할한 다음 진행한다. 다음은 SPECK의 암호화 라운드를 나타내는 수식이다.

$$\begin{aligned}x_{2i+3} &= (S^{-\alpha}(x_{2i+1}) + x_{2i}) \oplus k_i \\ x_{2i+2} &= S^{-\beta}(x_{2i}) \oplus x_{2i+3}\end{aligned}$$

α, β 는 블록 사이즈에 따라 값이 정해지며, 블록 사이즈가 128 bits인 경우 $\alpha = 8, \beta = 3$ 이다. 그림 15는 SPECK의 암호화 과정을 나타낸다.

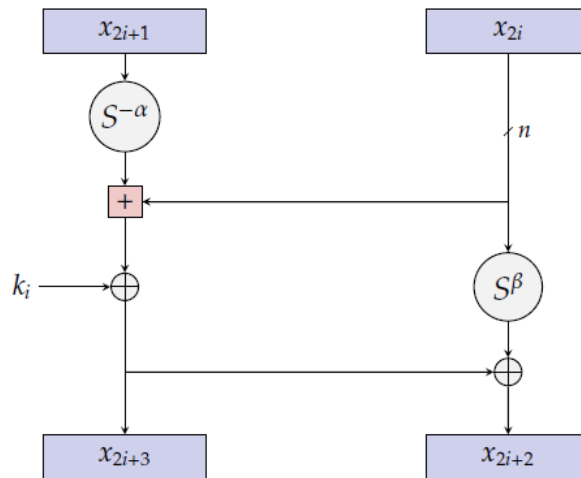


그림 15 - SPECK의 암호화 과정 (출처 : [9])

복호화 과정은 암호화 과정을 역으로 진행하면 된다.

NSA에서 제시한 SPECK 구현 명세서를 참고해 구현하였으며, 테스트 결과는 다음과 같다.

```
----- SPECK-128 Test -----  
  
key:  
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  
  
plainText:  
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  
  
cipherText:  
3D F1 AE B7 75 2D BB 06 C5 DE EA 14 3D A6 D6 30  
  
decryptedText:  
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

그림 16 - SPECK 테스트 결과, 암호화 및 복호화가 정상적으로 이루어짐을 보여준다.

4.1.5. TWINE

TWINE은 일본 전기 주식회사 NEC Corporation 소속의 연구원들이 발표한 경량 암호 알고리즘[10]이다. 이는 RFID나 Sensor Network Node와 같이 리소스가 매우 제한된 상황에서 동작하기 위해 고안된 블록 암호 알고리즘이다. 하드웨어 구현과 소프트웨어 구현 모두 효율적이며, 균형잡힌 성능을 제공한다. 블록 사이즈는 64 bits 이며 키 길이는 80, 128 bits 이다. 본 연구에서는 TWINE-128을 사용하였다.

TWINE의 Key Schedule은 상당히 복잡하지만, 암호화와 복호화에 같은 라운드 키를 사용한다. 총 라운드의 수가 36이며, 키 길이가 128 bits일 경우 16 bits씩 8 부분으로 나눈 뒤, 상위 4 bits와 하위 4 bits를 다시 나눈다. 이 중 특정 부분만 라운드 키가 되며, 계산을 통해 다음 라운드 키를 산출한다. 아래는 키 길이가 128 bits일 경우 Key Schedule을 하는 의사코드이다.

Algorithm A.1: TWINE.KeySchedule-128($K_{(128)}, RK_{(32 \times 36)}$)

```

WK(128) ←  $K$ 
WK0(16) || WK1(16) || ... || WK6(16) || WK7(16) ←  $WK$ 
RK0(4)1 ← WK2, RK1(4)1 ← WK3, RK2(4)1 ← WK12, RK3(4)1 ← WK15
RK4(4)1 ← WK17, RK5(4)1 ← WK18, RK6(4)1 ← WK28, RK7(4)1 ← WK31
RK(32)1 ← RK01 || RK11 || ... || RK61 || RK71
for  $i \leftarrow 2$  to 36
    WK1 ← WK1 ⊕  $S(WK_0)$ 
    WK4 ← WK4 ⊕  $S(WK_{16})$ 
    WK23 ← WK23 ⊕  $S(WK_{30})$ 
    WK7 ← WK7 ⊕ 0 || CONH $i-1$ 
    WK19 ← WK19 ⊕ 0 || CONL $i-1$ 
    tmp0 ← WK0, tmp1 ← WK1, tmp2 ← WK2, tmp3 ← WK3
    do
        for  $j \leftarrow 0$  to 6
            do
                WK $j*4$  ← WK $j*4+4$ , WK $j*4+1$  ← WK $j*4+5$ 
                WK $j*4+2$  ← WK $j*4+6$ , WK $j*4+3$  ← WK $j*4+7$ 
            WK28 ← tmp1, WK29 ← tmp2, WK30 ← tmp3, WK31 ← tmp0
            RK0 $i$  ← WK2, RK1 $i$  ← WK3, RK2 $i$  ← WK12, RK3 $i$  ← WK15
            RK4 $i$  ← WK17, RK5 $i$  ← WK18, RK6 $i$  ← WK28, RK7 $i$  ← WK31
            RK(32) $i$  ← RK0(4) $i$  || RK1(4) $i$  || ... || RK6(4) $i$  || RK7(4) $i$ 
        RK(32×36) ← RK(32)1 || RK(32)2 || ... || RK(32)35 || RK(32)36

```

CON은 라운드 상수로서 오른쪽에서부터 6 bits를 사용한다. 이는 원시 다항식 $z^6 + z + 1$ 을 갖는 $GF(2^6)$ 위의 값이며, 상위 3 bits와 하위 3 bits를 나눠 쓴다. 이를 수식으로 표현하면 다음과 같다.

$$CON_{(6)}^i = CON_{H(3)}^i \parallel CON_{L(3)}^i$$

암호화 라운드는 평문 64 bits를 4 bits의 서브 블록 16개로 분할한 뒤 작동한다. 아래는 TWINE의 암호화 과정을 나타내는 의사코드이다.

Algorithm 2.1: TWINE.Enc($P_{(64)}, RK_{(32 \times 36)}, C_{(64)}$)

```

X(64)1 ←  $P$ 
RK(32)1 || ... || RK(32)35 ←  $RK_{(32 \times 36)}$ 
for  $i \leftarrow 1$  to 35
    X0(4) $i$  || X1(4) $i$  || ... || X14(4) $i$  || X15(4) $i$  ← X(64) $i$ 
    RK0(4) $i$  || RK1(4) $i$  || ... || RK6(4) $i$  || RK7(4) $i$  ← RK(32) $i$ 
    for  $j \leftarrow 0$  to 7
        do X2j+1 $i$  ←  $S(X_{2j}^i \oplus RK_j^i) \oplus X_{2j+1}^i$ 
        for  $h \leftarrow 0$  to 15
            do X $\pi[h]$  $i+1$  ← X $h$  $i$ 
        X0 $i+1$  ← X0 $i+1$  || X1 $i+1$  || ... || X14 $i+1$  || X15 $i+1$ 
    for  $j \leftarrow 0$  to 7
        do X2j+136 ←  $S(X_{2j}^{36} \oplus RK_j^{36}) \oplus X_{2j+1}^{36}$ 
    C ← X036

```

각 라운드의 마지막 과정인 $X_{\pi[h]}^{i+1}$ 는 Block Shuffle을 나타내며, 이는 단순히 각 서브 블록을 섞는 것이다. 그림 17은 해당 과정을 그림으로 나타낸 것이며, 표 2는 해당 과정을 보기 쉽게 정리한 것이다.

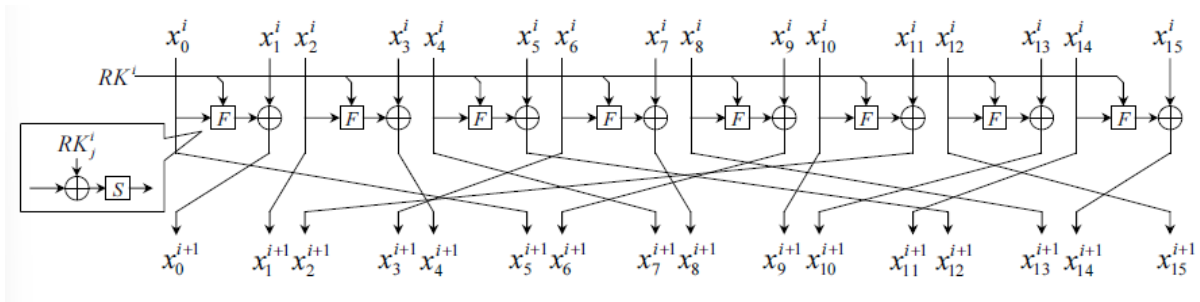


그림 17 - TWINE의 Block Shuffle 과정 (출처 : [10])

j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\pi[j]$	5	0	1	4	7	12	3	8	13	6	9	2	15	10	11	14
$\pi^{-1}[j]$	1	2	11	6	3	0	9	4	7	10	13	14	5	8	15	12

표 2 - TWINE의 Block Shuffle 과정 정리 (출처 : [10])

복호화 과정은 암호화 과정을 역으로 계산하면 된다.

연구자들이 제시한 TWINE의 논문을 참고해 구현하였으며, 테스트 결과는 다음과 같다.

```

----- TWINE-128 Test -----

key:
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

plainText:
00 00 00 00 00 00 00 00

cipherText:
CC 61 C7 79 DB D8 D2 42

decryptedText:
00 00 00 00 00 00 00 00

```

그림 18 - TWINE 테스트 결과, 암호화 및 복호화가 정상적으로 이루어짐을 보여준다.

4.2. 블록 암호 운영모드

블록 암호 운영모드는 어떤 블록 암호 알고리즘이 사용되더라도 일관된 인터페이스를 제공해야 한다. 따라서 앞서 한 블록 암호 알고리즘의 인터페이스와 유사하게 아래 인터페이스를 상속받고, 구현하는 식으로 진행하였다. 아래 인터페이스는 블록 암호 알고리즘을 생성자의 파라미터로 받는다.

```

public interface IOperationMode
{
    byte[] Encrypt(byte[] plainText);
    byte[] Decrypt(byte[] cipherText);
    string ModeName { get; }
}

```

```

        string AlgorithmName { get; }
    }

    public abstract class OperationMode(IEncryptionAlgorithm encryptionAlgorithm) : IOperationMode
    {
        public abstract byte[] Encrypt(byte[] plainText);
        public abstract byte[] Decrypt(byte[] cipherText);
        protected byte[] Padding(byte[] input)
        {
            int blockSize = encryptionAlgorithm.GetBlockSize();
            int paddingSize = blockSize - (input.Length % blockSize);
            byte[] output = new byte[input.Length + paddingSize];
            Array.Copy(input, output, input.Length);
            for (int i = input.Length; i < output.Length; i++)
            {
                output[i] = (byte)paddingSize;
            }

            return output;
        }
        protected byte[] UnPadding(byte[] input)
        {
            int paddingSize = input[input.Length - 1];
            for (int i = input.Length - paddingSize; i < input.Length; i++)
            {
                if (input[i] != paddingSize)
                    throw new ArgumentException("Invalid padding.");
            }
            byte[] output = new byte[input.Length - paddingSize];
            Array.Copy(input, output, output.Length);

            return output;
        }
        public abstract string ModeName { get; }
        public abstract string AlgorithmName { get; }

        protected readonly IEncryptionAlgorithm encryptionAlgorithm = encryptionAlgorithm;
    }

```

ECB를 제외한 나머지 운영모드의 경우 IV(Initialization Vector)가 필요하며, 각 테스트의 입력으로 IV가 주어지기에 위 인터페이스를 상속받은 클래스는 테스트에 필요한 함수를 따로 구현할 필요가 있다. GCM의 경우 IV외에 추가로 AAD(Additional Authenticated Data)가 필요하며, 암호화의 결과로 태그(Auth Tag)가 생성된다. 또한, GCM은 암호화할 평문이 없더라도 태그 계산이 가능하기에 해당 특성들을 고려하여 설계할 필요가 있다.

패딩이 필요한 운영모드(ECB, CBC)에 적용하기 위해 Padding, UnPadding 함수를 추가하였다. 이는 PKCS#7 패딩 기법이며, 블록 암호 알고리즘의 블록 사이즈에 맞춰 패딩을 할 수 있도록 도와준다. 그림 19는 블록 암호 운영모드의 클래스 다이어그램이다.

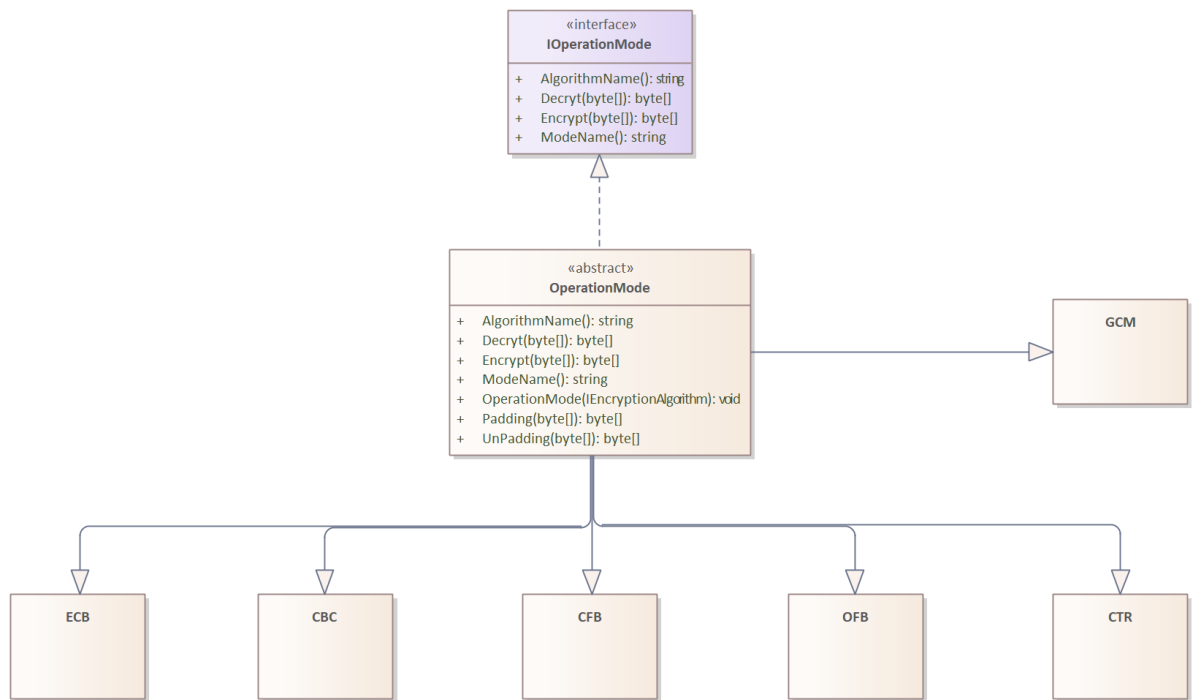


그림 19 - 블록 암호 운영모드의 클래스 다이어그램

4.2.1. ECB

ECB(Electronic Code Book)는 가장 단순한 운영모드로, 데이터를 블록 단위로 순차적으로 암호화 하는 구조이다. 각 블록은 이전 블록의 상태에 영향을 받지 않으므로 병렬 처리가 가능하다는 장점이 있다. 그러나 하나의 블록이 해독될 경우 같은 키를 사용하는 나머지 블록도 모두 해독될 수 있어 보안상 권장되지 않는 모드이다. ECB 모드는 평문의 길이가 블록 사이즈의 배수가 아닐 경우 패딩이 필요하다. 그림 20은 ECB 모드의 암호화 동작 과정을 나타내는 그림이다.

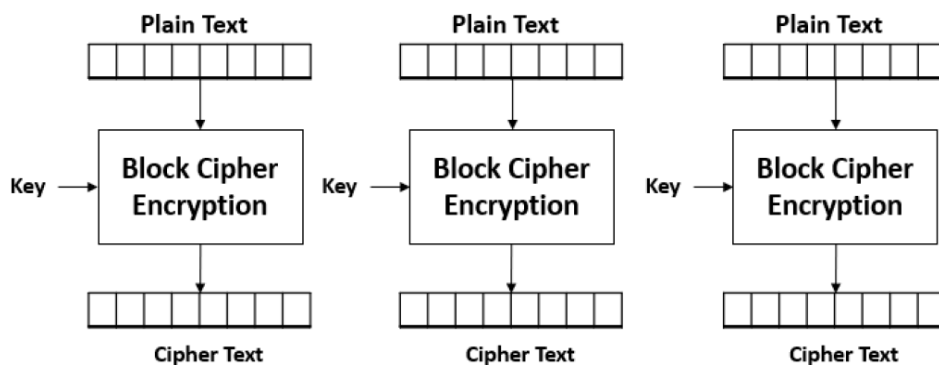


그림 20 - ECB 모드의 암호화 동작 과정 (출처 : [11])

암호화 함수를 E_k , 복호화 함수를 D_k , 평문과 암호문의 i^{th} 블록을 각각 P_i , C_i 라 할 때, 암호화와 복호화 과정을 수식으로 표현하면 다음과 같다.

$$C_i = E_k(P_i)$$

$$P_i = D_k(C_i)$$

4.2.2. CBC

CBC(Cipher Block Chaining)는 앞서 진행한 ECB와는 다르게, 평문 블록을 이전 암호문 블록과 XOR하며 진행한다. 첫 블록 암호화는 IV가 필요하며, 각 블록이 이전 블록의 상태에 영향을 받기에 병렬 처리가 불가능하다. ECB와 마찬가지로 평문의 길이가 블록 사이즈의 배수가 아닐 경우 패딩이 필요하다. 그림 21은 CBC 모드의 암호화 동작 과정을 나타낸다.

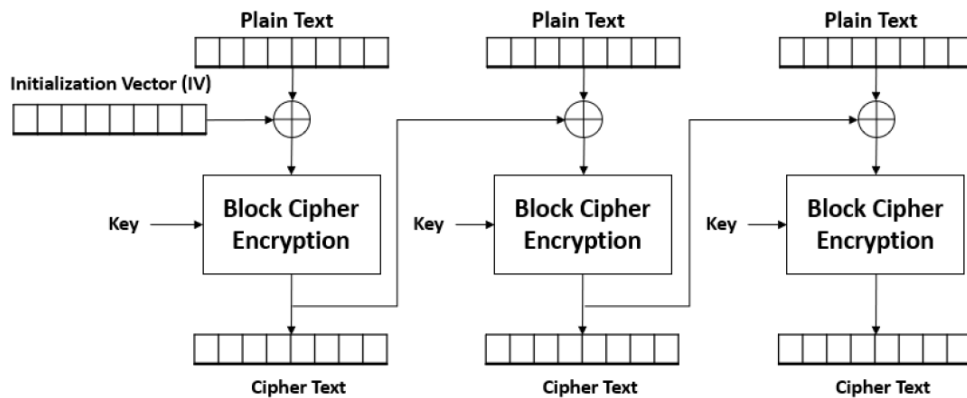


그림 21 - CBC 모드의 암호화 동작 과정 (출처 : [11])

암호화 함수를 E_k , 복호화 함수를 D_k , 평문과 암호문의 i^{th} 블록을 각각 P_i , C_i 라 할 때, 암호화와 복호화 과정을 수식으로 표현하면 다음과 같다.

$$C_i = E_k(P_i \oplus C_{i-1}), C_0 = IV$$

$$P_i = D_k(C_i) \oplus C_{i-1}, C_0 = IV$$

4.2.3. CFB

CFB(Cipher Feedback)는 평문을 암호화하는 시점이 앞의 두 모드와 다르다. CFB는 평문을 이전 암호문 블록의 암호화 값과 XOR하여 암호문 블록을 생성한다. 따라서, 해당 모드에서는 패딩이 필요 없으며, 블록 암호를 스트림 암호처럼 다룰 수 있게 해준다. 그림 22는 CFB 모드의 암호화 동작 과정을 나타낸다.

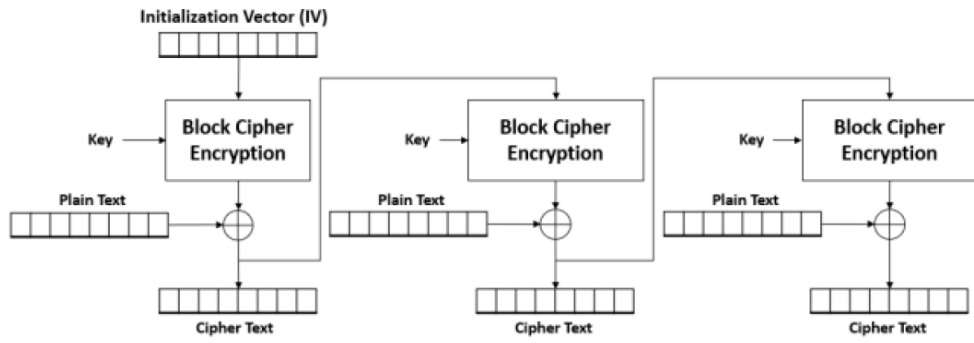


그림 22 - CFB 모드의 암호화 동작 과정 (출처 : [11])

앞서 본 모드와 다르게 복호화 함수가 따로 필요하지 않다. 암호화 함수를 E_k , 평문과 암호문의 i^{th} 블록을 각각 P_i, C_i 라 할 때, 암호화와 복호화 과정을 수식으로 표현하면 다음과 같다.

$$C_i = E_k(C_{i-1}) \oplus P_i, C_0 = IV$$

$$P_i = E_k(C_{i-1}) \oplus C_i, C_0 = IV$$

4.2.4. OFB

OFB(Output Feedback)는 CFB와 구조가 유사하나, OFB는 암호 알고리즘을 적용한 블록이 바로 다음 블록의 암호화에 영향을 준다는 점에서 다르다. CFB와 마찬가지로 패딩이 필요 없으며 스트림 암호처럼 다룰 수 있다. 그림 23은 OFB 모드의 암호화 동작 과정을 나타낸다.

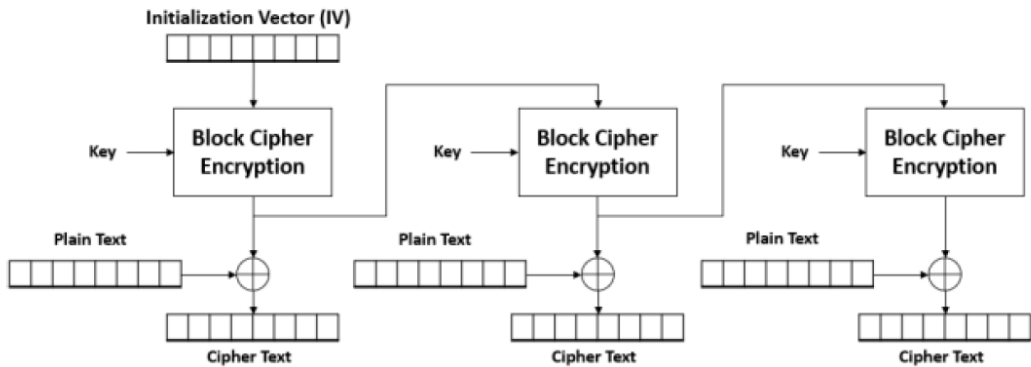


그림 23 - OFB 모드의 암호화 동작 과정 (출처 : [11])

OFB 모드 또한 CFB 모드와 마찬가지로 복호화 함수가 따로 필요하지 않다. 암호화 함수를 E_k , 평문과 암호문의 i^{th} 블록을 각각 P_i, C_i , Output의 i^{th} 블록을 O_i 라 할 때, 암호화와 복호화 과정을 수식으로 표현하면 다음과 같다.

$$O_i = E_k(O_{i-1}), O_0 = IV$$

$$C_i = P_i \oplus O_i$$

$$P_i = C_i \oplus O_i$$

4.2.5. CTR

CTR(Counter)는 CFB, OFB 모드와 동작이 매우 유사하다. 다른 점은, CTR 모드는 암호화 함수의 입력값에 Counter 블록을 준다는 점이다. 또한 이전 블록이 현재 블록에 영향을 주지 않기 때문에 암호화 및 복호화 모두 병렬 처리가 가능하다. Counter 블록은 블록 크기와 같은 Nonce가 정해지면, 해당 값에서 1씩 증가시키는 형태로 진행한다. 이는 앞서 본 CBC, CFB, OFB의 IV와 같은 역할이다. 그림 24은 CTR 모드의 암호화 동작 과정을 나타낸다.

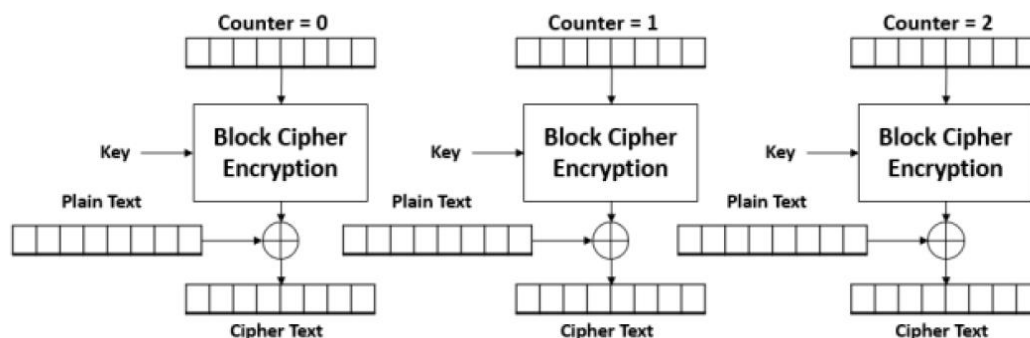


그림 24 - CTR 모드의 암호화 동작 과정 (출처 : [11])

CTR 모드 또한 복호화 함수가 따로 필요하지 않다. 암호화 함수를 E_k , 평문과 암호문의 i^{th} 블록을 각각 P_i , C_i , Counter의 i^{th} 블록을 Ctr_i 라 할 때, 암호화와 복호화 과정을 수식으로 표현하면 다음과 같다.

$$\text{Ctr}_i = \text{Ctr}_{i-1} + 1, \text{Ctr}_0 = \text{Nonce}$$

$$C_i = E(\text{Ctr}_i) \oplus P_i$$

$$P_i = E(\text{Ctr}_i) \oplus C_i$$

4.2.6. GCM

GCM(Galois/Counter Mode)은 앞서 본 모드들과는 다르게 메시지의 기밀성 뿐만 아니라 무결성과 신뢰성 또한 보장할 수 있다. NIST 표준에 등록되어 있으며 [12], CTR 모드를 통해 메시지를 암호화 하면서도 GMAC(Galois Message Authentication Code)를 통해 메시지의 인증과 동시에 무결성을 확인할 수 있다. 그림 25는 GCM 모드의 암호화 동작 과정을 나타낸다.

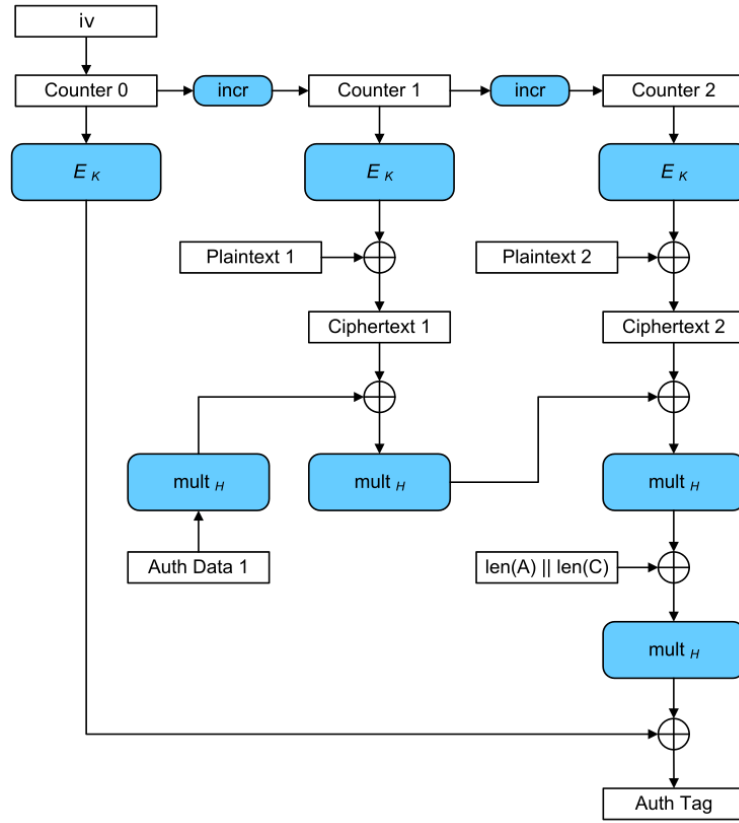


그림 25 - GCM 모드의 암호화 동작 과정 (출처 : [13])

GCM 모드는 크게 평문을 암호화 하는 부분과, Auth Tag를 계산하는 부분으로 나뉜다. 앞서 본 모드들과 다르게 IV의 길이가 블록 사이즈와 같을 필요는 없으며, IV의 길이가 96 bits일 때와 아닐 때 Counter 블록의 계산 방법이 달라진다.

평문을 암호화 하는 부분은 앞서 한 CTR 모드와 같다. 하지만, IV에서 Counter 블록을 계산하는 부분이 다르다. 아래 수식은 Counter 블록 Y_0 의 생성 방법이다.

$$H = E(K, 0^{128})$$

$$Y_0 = \begin{cases} IV \parallel 0^{31} & \text{if } \text{len}(IV) = 96 \\ GHASH(H, \{\}, IV) & \text{otherwise} \end{cases}$$

GHASH는 내부적으로 사용되는 해시 함수로, 입력 A, C에 대해 $GHASH(H, A, C) = X_{m+n+1}$ 는 다음과 같이 정의된다.

$$X_i = \begin{cases} 0 & \text{for } i = 0 \\ (X_{i-1} \oplus A_i) \cdot H & \text{for } i = 1, \dots, m-1 \\ (X_{m-1} \oplus (A_m^* \parallel 0^{128-v})) \cdot H & \text{for } i = m \\ (X_{i-1} \oplus C_{i-m}) \cdot H & \text{for } i = m+1, \dots, m+n-1 \\ (X_{m+n-1} \oplus (C_n^* \parallel 0^{128-u})) \cdot H & \text{for } i = m+n \\ (X_{m+n} \oplus (\text{len}(A) \parallel \text{len}(C))) \cdot H & \text{for } i = m+n+1. \end{cases}$$

이때 입력 A는 $A_0, A_1, A_2, \dots, A_{m-1}, A_m^*$ 로 표현된다. A_i 는 AAD를 암호 알고리즘의 블록 사이즈

단위로 분할 했을 때 i^{th} 블록을 의미하며, A_m^* 은 마지막 블록을 의미한다. 입력 C 또한 동일하게 적용된다.

GHASH를 계산하기 위해선 Galois Field 위에서의 곱셈이 필요하다. GCM은 원시 방정식 $f = 1 + \alpha + \alpha^2 + \alpha^7 + \alpha^{128}$ 을 사용하는 $GF(2^{128})$ 위에서 동작한다. 아래는 입력 X, Y 에 대해 $Z = X \cdot Y$, where $X, Y, Z \in GF(2^{128})$ 을 계산하는 의사코드이다.

Algorithm 1 Multiplication in $GF(2^{128})$. Computes the value of $Z = X \cdot Y$, where X, Y and $Z \in GF(2^{128})$.

```

 $Z \leftarrow 0, V \leftarrow X$ 
for  $i = 0$  to 127 do
  if  $Y_i = 1$  then
     $Z \leftarrow Z \oplus V$ 
  end if
  if  $V_{127} = 0$  then
     $V \leftarrow \text{rightshift}(V)$ 
  else
     $V \leftarrow \text{rightshift}(V) \oplus R$ 
  end if
end for
return  $Z$ 

```

$\text{rightshift}(V)$ 는 V 를 1 bit씩 오른쪽으로 shift하는 연산이다.

위 과정을 통해 Counter 블록을 생성했다면, 해당 Counter 블록을 토대로 암호화와 태그 계산을 진행하면 된다. 평문과 암호문의 i^{th} 블록을 P_i, C_i 라 할 때 암호화 과정을 나타낸 수식은 다음과 같다.

$$\begin{aligned}
 Y_i &= \text{incr}(Y_{i-1}), & \text{for } i = 1, \dots, n \\
 C_i &= P_i \oplus E(K, Y_i), & \text{for } i = 1, \dots, n-1 \\
 C_n^* &= P_n^* \oplus \text{MSB}_u(E(K, Y_n))
 \end{aligned}$$

이때 MSB_u 는 Most Significant Bit(leftmost bit) 이며, u 는 C_n^* 의 마지막 bit 수를 의미한다. 태그 계산은 보통 암호화 과정과 같이 진행하며, 태그 T 의 계산 과정을 나타낸 수식은 다음과 같다.

$$T = \text{MSB}_t(\text{GHASH}(H, A, C) \oplus E(K, Y_0))$$

A 는 AAD(Additional Authenticated data)로 평문 외에 추가적인 인증 정보로 사용한다.

복호화 과정의 경우 암호화 과정과 대부분 동일하며, 암호문을 복호화하는 과정 또한 CTR 모드의 복호화 과정과 동일하다.

4.2.7. 테스트

AES 테스트는 NIST에서 제시한 CAVP(Cryptographic Algorithm Validation Program)의 테스트 벡터[14]를 사용하였다. ARIA와 HIGHT의 테스트는 KISA에서 제공한 테스트 벡터[15]를 활용하였다. SPECK 테스트는 구현 명세서에 포함된 ECB 테스트 벡터 외에도 추가적인 테스트

벡터를 깃허브에서 찾을 수 있었으며, 이 벡터들은 ECB, CBC, CTR 모드의 테스트만 지원한다. TWINE 테스트는 구현 명세서에 포함된 ECB 테스트 벡터 외에는 다른 테스트 벡터가 없어 ECB 모드에서만 테스트를 진행하였다. 표 3은 각 테스트 모드에 대한 통과 판정 조건을 보여준다.

테스트 모드	통과 판정 조건
ECB	주어진 키에 대해 1) 평문을 암호화한 것이 암호문과 같은가 2) 암호문을 복호화한 것이 평문과 같은가
CBC, CFB, OFB, CTR	주어진 키와 IV에 대해 1) 평문을 암호화한 것이 암호문과 같은가 2) 암호문을 복호화한 것이 평문과 같은가
GCM_Encrypt	주어진 키, IV, AAD, 태그에 대해 1) 평문을 암호화한 것이 암호문과 같은가 2) 암호문을 복호화한 것이 평문과 같은가 3) 태그를 계산한 것이 테스트 벡터의 태그와 같은가
GCM_Decrypt	주어진 키, IV, AAD, 태그 판정 결과에 대해 1) 정상 태그로 판정했을 때 태그 판정 결과가 정상인가 2) 비정상 태그로 판정했을 때 태그 판정 결과가 비정상인가 3) 정상 태그일 때 평문을 암호화한 것이 암호문과 같은가 4) 정상 태그일 때 암호문을 복호화한 것이 평문과 같은가

표 3 - 테스트 모드의 통과 판정 조건

아래는 각 블록 암호 알고리즘의 운영모드별 테스트 실행 화면이다.

```

***** AES TEST START *****

[AES - ECB] [Success] ECBGFSbox128.txt
[AES - ECB] [Success] ECBKeySbox128.txt
[AES - ECB] [Success] ECBMMT128.txt
[AES - ECB] [Success] ECBVarKey128.txt
[AES - ECB] [Success] ECBVarTxt128.txt

[AES - CBC] [Success] CBCGFSbox128.txt
[AES - CBC] [Success] CBCKeySbox128.txt
[AES - CBC] [Success] CBCMMT128.txt
[AES - CBC] [Success] CBCVarKey128.txt
[AES - CBC] [Success] CBCVarTxt128.txt

[AES - CFB] [Success] CFB128GFSbox128.txt
[AES - CFB] [Success] CFB128KeySbox128.txt
[AES - CFB] [Success] CFB128MMT128.txt
[AES - CFB] [Success] CFB128VarKey128.txt
[AES - CFB] [Success] CFB128VarTxt128.txt

[AES - OFB] [Success] OFBGFSbox128.txt
[AES - OFB] [Success] OFBKeySbox128.txt
[AES - OFB] [Success] OFBMMT128.txt
[AES - OFB] [Success] OFBVarKey128.txt
[AES - OFB] [Success] OFBVarTxt128.txt

[AES - CTR] [Success] CTR128.txt

[AES - GCM] [Success] gcmDecrypt128.txt
[AES - GCM] [Success] gcmEncryptExtIV128.txt

***** AES TEST END *****

```

그림 26 - AES 테스트 실행 화면, 각 운영모드의 테스트를 통과한 모습이다.

```

***** ARIA TEST START *****

[ARIA - ECB] [Success] ARIA128(ECB)KAT.txt
[ARIA - ECB] [Success] ARIA128(ECB)MMT.txt

[ARIA - CBC] [Success] ARIA128(CBC)KAT.txt
[ARIA - CBC] [Success] ARIA128(CBC)MMT.txt

[ARIA - CFB] [Success] ARIA128(CFB128)KAT.txt
[ARIA - CFB] [Success] ARIA128(CFB128)MMT.txt

[ARIA - OFB] [Success] ARIA128(OFB)KAT.txt
[ARIA - OFB] [Success] ARIA128(OFB)MMT.txt

[ARIA - CTR] [Success] ARIA128(CTR)KAT.txt
[ARIA - CTR] [Success] ARIA128(CTR)MMT.txt

[ARIA - GCM] [Success] ARIA128(GCM)VTDF.txt
[ARIA - GCM] [Success] ARIA128(GCM)VTEF.txt

***** ARIA TEST END *****

```

그림 27 - ARIA 테스트 실행 화면, 각 운영모드의 테스트를 통과한 모습이다.

```

***** HIGHT TEST START *****

[HIGHT - ECB] [Success] HIGHT(ECB)KAT.txt
[HIGHT - ECB] [Success] HIGHT(ECB)MMT.txt

[HIGHT - CBC] [Success] HIGHT(CBC)KAT.txt
[HIGHT - CBC] [Success] HIGHT(CBC)MMT.txt

[HIGHT - CFB] [Success] HIGHT(CFB64)KAT.txt
[HIGHT - CFB] [Success] HIGHT(CFB64)MMT.txt

[HIGHT - OFB] [Success] HIGHT(OFB)KAT.txt
[HIGHT - OFB] [Success] HIGHT(OFB)MMT.txt

[HIGHT - CTR] [Success] HIGHT(CTR)KAT.txt
[HIGHT - CTR] [Success] HIGHT(CTR)MMT.txt

[HIGHT - GCM] [Fail] There is no test vectors.

***** HIGHT TEST END *****

```

그림 28 - HIGHT 테스트 실행 화면, 각 운영모드의 테스트를 통과한 모습이다.

```

***** SPECK TEST START *****

[SPECK - ECB] [Success] SPECK128_ECB.txt
[SPECK - CBC] [Success] SPECK128_CBC.txt
[SPECK - CFB] [Fail] There is no test vectors.
[SPECK - OFB] [Fail] There is no test vectors.
[SPECK - CTR] [Success] SPECK128_CTR.txt
[SPECK - GCM] [Fail] There is no test vectors.

***** SPECK TEST END *****

```

그림 29 - SPECK 테스트 실행 화면, ECB, CBC, CTR 모드의 테스트를 통과한 모습이다

```
***** TWINE TEST START *****
[TWINE - ECB] [Success] TWINE128_ECB.txt
[TWINE - CBC] [Fail] There is no test vectors.
[TWINE - CFB] [Fail] There is no test vectors.
[TWINE - OFB] [Fail] There is no test vectors.
[TWINE - CTR] [Fail] There is no test vectors.
[TWINE - GCM] [Fail] There is no test vectors.
***** TWINE TEST END *****
```

그림 30 - TWINE 테스트 실행 화면, ECB 모드의 테스트를 통과한 모습이다.

표 4는 각 블록 암호 알고리즘과 운영모드별 테스트 통과 여부를 나타낸다.

	AES	ARIA	HIGHT	SPECK	TWINE
ECB	O	O	O	O	O
CBC	O	O	O	O	X
CFB	O	O	O	X	X
OFB	O	O	O	X	X
CTR	O	O	O	O	X
GCM	O	O	구현 X	X	구현 X

표 4 - 블록 암호 알고리즘과 운영모드별 테스트 통과 여부

위 과정을 통해 AES, ARIA, HIGHT는 모든 블록 암호 운영모드에 대해 표준 및 구현 명세서에 맞게 설계되었음을 확인할 수 있다. SPECK, TWINE 일부 운영모드에 대해 구현 명세서에 맞게 설계되었음을 확인할 수 있다.

4.3. ECC

ECC(Elliptic-Curve Cryptography)는 공개키 암호로서 유한체 위의 타원 곡선을 기반으로 한다. ECC를 활용하는 키 교환 알고리즘(ECDH)과 전자 서명 알고리즘(ECDSA)이 NIST 표준[16]으로 등록되었으며 ECC를 사용할 때 권장하는 파라미터 또한 표준[17]으로 등록되어 있다. ECC에 사용되는 파라미터는 a, b, p, n, G 이며 본 연구에서는 P-256을 사용하였고, 해시 함수는 SHA-256을 사용하였다. 각 파라미터에 대한 설명은 다음과 같다.

$$y^2 = x^3 + ax + b \bmod p$$

a : 타원 곡선 방정식의 계수 a이다. P-256에서 해당 값은 아래와 같다.

```
a = 0xffffffff00000001000000000000000000000000000000000000000000000000000000c
```

b : 타원 곡선 방정식의 상수 b이다. P-256에서 해당 값은 아래와 같다.

b = 5ac635d8aa3a93e7b3ebbd55769886bc651d06b0cc53b0f63bce3c3e27d2604b

p : 타원 곡선 방정식의 모듈러 공간을 결정한다. P-256에서 해당 값은 아래와 같다.

p = ffffffff000000010000000000000000000000000000000000ffffffffffffffffffffffff

n : 타원 곡선 방정식의 차수이다. 이는 $G * n = 0$ 이 되는 n 값이며 P-256에서 해당 값은 아래와 같다

```
n = ffffffff00000000ffffffffffffffbce6faada7179e84f3b9cac2fc632551.
```

G : 타원 곡선 방정식의 생성점(Generator)이다. P-256에서 해당 값은 아래와 같다.

Gx = 6b17d1f2e12c4247f8bce6e563a440f277037d812deb33a0f4a13945d898c296

Gy = 4fe342e2fe1a7f9b8ee7eb4a7c0f9e162bce33576b315ececbb6406837bf51f5

타원곡선상의 점을 표현하기 위해 아래의 ECPoint 클래스를 사용하였다.

```
public class ECPoint(BigInteger x, BigInteger y)
{
    public BigInteger X { get; } = x;
    public BigInteger Y { get; } = y;

    public bool IsInfinity() { return X == 0 && Y == 0; }

    public static readonly ECPoint Infinity = new(0, 0);
}
```

(0, 0)은 무한원점을 의미하고, 256 bits의 큰 수 연산을 위해 .NET의 내장 라이브러리인 BigInteger를 사용하였다. ECC 클래스는 ECPoint의 연산을 위해 다음과 같은 함수를 정의한다.

- AddPoints(ECPoint p1, ECPoint p2): 점 덧셈, $p1 + p2$
- MultiplyPoint(ECPoint point, BigInteger k): 점의 스칼라 곱셈, $point * k$
- ModInverse(BigInteger a, BigInteger m): mod m 공간에서의 역원, $a^{-1} \bmod m$

ECC는 비밀키로부터 공개키를 얻고, ECPoint의 유효성을 검증하기 위해 다음과 같은 함수를 정의한다.

- `GetPublicKey(BigInteger privKey)`: 비밀키로부터 공개키를 얻는다. $G * \text{privKey}$
- `IsValidPoint(ECPPoint p)`: p 가 타원 곡선 상에 존재하는지, mod n 공간에 존재하는지를 확인한다.

4.4. ECDSA

ECDSA(Elliptic-Curve Digital Signature Algorithm)는 NIST FIPS 186-4에 표준[16]으로 등록되어 있는 타원 곡선을 활용한 전자 서명 알고리즘이다. ECDSA는 크게 메시지에 대한 서명 부분과 서명에 대한 검증 부분으로 나뉜다. ECDSA 클래스는 서명의 생성과 검증을 위해 다음과 같은 함수를 정의한다.

- `Sign(byte[] message, BigInteger privKey)` : 서명 (r, s)를 생성한다.
- `Verify(byte[] message, byte[] signature, ECPoint pubKey)` : 서명 (r, s)를 검증한다.

서명 (r, s)를 생성하는 방식은 다음과 같다.

- 1) $z = \text{Hash}(\text{message})$
- 2) $R = G * k$ 인 $\{1, \dots, n-1\}$ 사이의 랜덤한 수 k 를 선택한다.
- 3) $R.x = 0$ 일 경우 1번으로 돌아간다.
- 4) $r = R.x$ 에 대해 $s = k^{-1} * (z + r * \text{privKey})$

서명 (r, s)를 검증하는 방식은 다음과 같다.

- 1) $z = \text{Hash}(\text{message})$
- 2) $0 < r < n \ \&\& \ 0 < s < n$ 확인한다.
- 3) $w = s^{-1}$ 에 대해 $u1 = z * w$, $u2 = r * w$
- 4) $\text{point1} = G * u1$, $\text{point2} = \text{pubKey} * u2$ 에 대해 $R = \text{point1} + \text{point2}$
- 5) R이 무한 원점인지 확인한다.
- 6) $R.x == r$ 인지 확인한다.

ECDSA의 테스트는 NIST에서 제시한 ECDSA2VS의 테스트 벡터[14]를 사용하였다. ECDSA의 테스트는 크게 4가지가 있으며 표 5는 각 테스트와 테스트 별 통과 판정 조건을 보여준다.

테스트	통과 판정 조건
KeyPairTest	비밀키로 생성되는 공개키가 정상적인지 테스트한다. 주어진 비밀키(d), 공개키(Qx, Qy)에 대해 1) d를 통해 생성된 공개키가 Qx, Qy와 같은가
PKVTest	공개키가 정상적인지 판별하는 부분을 테스트한다. 주어진 공개키(Qx, Qy), 판별 결과(Result)에 대해 1) Qx, Qy가 정상적인 공개키라 판별했을 때 Result가 P인가 2) Qx, Qy가 비정상적인 공개키라 판별했을 때 Result가 F인가
SigGenTest	서명 생성을 테스트한다. 주어진 메시지(Msg), 비밀키(d), 공개키(Qx, Qy), 랜덤값(k), 서명 (R, S)에 대해 1) Msg, d, k로 생성된 서명이 (R, S)와 같은가
SigVerTest	서명 검증을 테스트한다. 주어진 메시지(Msg), 공개키(Qx, Qy), 서명 (R, S), 판별 결과(Result)에 대해 1) 서명 (R, S)가 정상이라 판단했을 때 Result 또한 P인가 2) 서명 (R, S)가 비정상이라 판단했을 때 Result 또한 F인가

표 5 - ECDSA 테스트와 통과 판정 조건

아래는 ECDSA의 테스트 실행 화면이다.

```

***** ECDSA(P-256,SHA-256) TEST START *****

[ECDSA - KeyPairTest]    [Success]    KeyPairTest.txt
[ECDSA - PKVTest]       [Success]    PKVTest.txt
[ECDSA - SigVerTest]    [Success]    SigVerTest.txt
[ECDSA - SigGenTest]    [Success]    SigGenTest.txt

***** ECDSA(P-256,SHA-256) TEST END *****

```

그림 31 - ECDSA 테스트 실행 화면, 각 테스트를 통과한 모습이다.

위 과정을 통해 ECDSA가 표준에 맞게 구현되었음을 확인할 수 있다.

4.5. ECDH

ECDH(Elliptic-Curve Diffie-Hellman)는 NIST SP 800-56A[18]에 표준으로 등록되어 있는 타원 곡선을 활용한 키 교환 알고리즘이다. ECDH는 통신중인 두 당사자가 안전하게 공유 비밀을 생성하기 위해 고안되었으며 타원 곡선상의 점의 스칼라 곱셈을 이용한다. 그림 32는 ECDH 과정을 나타낸다.

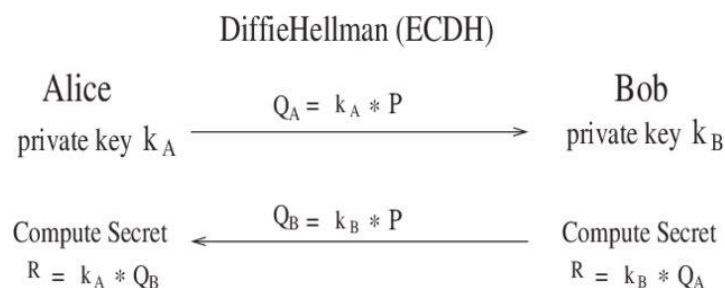


그림 32 - ECDH 과정

Alice와 Bob은 비밀키를 통해 공개키를 생성하고, 해당 공개키를 상대방에게 넘김으로써 공유 비밀을 계산한다. 이때 공유 비밀은 ECPoint.X를 사용한다. 즉, ECDH는 단순히 ECC에서 MultiplyPoint 함수만 사용하면 된다. ECDH 클래스는 공유 비밀의 계산을 위해 다음과 같은 함수를 정의한다.

- ComputeSharedSecret(BigInteger privKey, ECPoint pubKey): 공유 비밀을 계산한다.

ECDH의 테스트는 NIST에서 제시한 ECC_CDHVS의 테스트 벡터[14]를 사용하였다. ECDH의 테스트는 하나이며 표 6은 테스트의 종류와 통과 판정 조건을 보여준다.

테스트	통과 판정 조건
IUTTest	구현된 알고리즘이 정상 동작 하는지 테스트한다. 주어진 상대방의 공개키(QCAVSx, QCAVSy), 자신의 비밀키(diUT), 자신의 공개키(QIUTx, QIUTy), 공유 비밀(ZIUT)에 대해 1) diUT에 계산된 공개키가 QIUTx, QIUTy와 같은가 2) QCAVSx, QCAVSy, diUT를 통해 계산된 공유 비밀이 ZIUT와 같은가

표 6 - ECDH의 테스트와 통과 판정 조건

아래는 ECDH 테스트의 실행 화면이다.

```
***** ECDH(P-256) TEST START *****
[ECDH - IUTTest]           [Success]           ECDHTestVectorP256.txt
***** ECDH(P-256) TEST END *****
```

그림 33 - ECDH 테스트 실행 화면, 테스트를 통과한 모습이다.

위 과정을 통해 ECDH가 표준에 맞게 구현되었음을 알 수 있다.

4.6. HKDF

HKDF(HMAC-based Key Derivation Function)는 키 유도 함수로 임의의 길이의 입력값에서 고정된 길이의 암호 키를 생성하는 데 사용되는 함수이다. RFC5869[19]에 기술되어 있으며, NIST SP 800-56C[20]에 표준으로 등록되어 있다. HKDF는 이를 위해 HMAC_Hash를 사용하며, 크게 Extract 단계와 Expand 단계로 구분된다.

HMAC_Hash는 해시 함수를 통해 MAC(Message Authentication Code)를 계산하는 알고리즘으로, RFC 2104[21]에 기술되어 있으며, 본 연구에서는 이를 참고하여 구현하였다. 사용한 해시 함수는 SHA-256이며, 함수의 프로토타입은 다음과 같다.

```
byte[] HMAC_Hash(byte[] key, byte[] message)
```

Extract 단계는 입력 키(IKM, Input Key Management)로부터 PRK(Pseudo-Random Key)를 추출하는 단계이다. 이때 salt를 사용하여 랜덤성을 부여할 수 있으며, 수식과 함수의 프로토타입은 다음과 같다.

$$PRK = \text{HMAC_Hash}(\text{salt}, \text{IKM})$$

```
byte[] KeyExtract(byte[]? salt, byte[] IKM) -> PRK
```

이때 salt가 없을 경우 salt는 모든 비트가 0인 상태로 입력이 된다. 여기서 주의할 점은 salt가 HMAC_Hash의 key 값이 된다는 점이다.

Expand 단계는 PRK에서 원하는 길이의 최종 출력 키(OKM, Output Key Material)를 생성하는 단계이다. 해당 단계의 의사코드와 함수 프로토타입은 다음과 같다.

$$N = \text{ceil}(L/\text{HashLen})$$
$$T = T(1) \parallel T(2) \parallel T(3) \parallel \dots \parallel T(N)$$
$$T(0) = \text{empty string (zero length)}$$
$$T(1) = \text{HMAC - Hash}(\text{PRK}, T(0) \parallel \text{info} \parallel 0x01)$$
$$T(2) = \text{HMAC - Hash}(\text{PRK}, T(1) \parallel \text{info} \parallel 0x02)$$
$$T(3) = \text{HMAC - Hash}(\text{PRK}, T(2) \parallel \text{info} \parallel 0x03)$$

...

OKM = first L octets of T

`byte[] KeyExpand(byte[] PRK, byte[]? info, int L) -> OKM`

info는 부가 정보이며 없으면 무시된다. HashLen은 사용된 해시 함수의 결과 길이이며, Byte 단위이다. 이는 SHA-256 기준 32이다.

Extract 단계와 Expand 단계를 종합하여 키 유도가 이루어진다. 본 연구에서 사용한 함수는 다음과 같다.

```
public static byte[] KeyDerivation(byte[] sharedSecret, byte[] salt, byte[]? info, int L)
{
    byte[] temp = KeyExtract(salt, sharedSecret);
    byte[] ret = KeyExpand(temp, info, L);
    return ret;
}
```

HKDF 테스트는 RFC5869에 있는 테스트 벡터[19]를 사용하였다. 총 7개의 테스트 벡터가 존재하며, 아래는 테스트 벡터 1번의 실행 결과이다.

```
IKM:
0B 0B 0B 0B 0B 0B 0B 0B 0B 0B 0B 0B 0B 0B 0B 0B 0B 0B 0B 0B

salt:
00 01 02 03 04 05 06 07 08 09 0A 0B 0C

info:
F0 F1 F2 F3 F4 F5 F6 F7 F8 F9

L :
42

PRK :
07 77 09 36 2C 2E 32 DF 0D DC 3F 0D C4 7B BA 63 90 B6 C7 3B B5 0F 9C 31 22
EC 84 4A D7 C2 B3 E5

TestVectorPRK :
07 77 09 36 2C 2E 32 DF 0D DC 3F 0D C4 7B BA 63 90 B6 C7 3B B5 0F 9C 31 22
EC 84 4A D7 C2 B3 E5

OKM
3C B2 5F 25 FA AC D5 7A 90 43 4F 64 D0 36 2F 2A 2D 2D 0A 90 CF 1A 5A 4C 5D
B0 2D 56 EC C4 C5 BF 34 00 72 08 D5 B8 87 18 58 65

TestVectorOKM :
3C B2 5F 25 FA AC D5 7A 90 43 4F 64 D0 36 2F 2A 2D 2D 0A 90 CF 1A 5A 4C 5D
B0 2D 56 EC C4 C5 BF 34 00 72 08 D5 B8 87 18 58 65
```

그림 34 - HKDF 테스트 실행 화면, PRK와 OKM이 테스트 벡터의 것과 동일하다.

위 과정을 통해 HKDF가 표준에 맞게 구현되었음을 알 수 있다.

4.7. Handshake Protocol

Handshake Protocol은 최초 TCP 세션 수립 후 공유 비밀을 생성하고, 키 유도 함수를 통해 대칭키 암호에 사용할 암호 키를 만드는 과정이다. TLS 1.3을 참고하여 설계하였다. 그림 35는

본 연구에서 사용한 Handshake Protocol의 과정을 나타낸다.

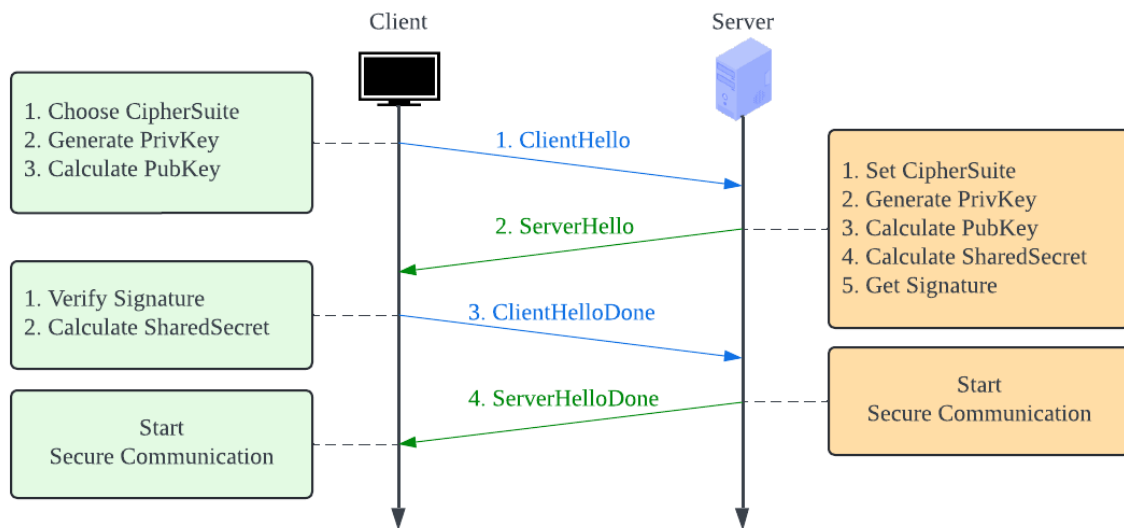


그림 35 - Handshake Protocol 과정

핸드셰이크 과정에서 전자 서명 알고리즘으로 ECDSA, 키 교환 알고리즘으로 ECDH, 키 유도 함수로 HKDF가 사용되었다. 클라이언트를 A, 서버를 B라 할 때 각 단계별 설명은 다음과 같다.

1) ClientHello

클라이언트는 핸드셰이크가 완료된 후 사용할 대칭 키 암호 알고리즘(Cipher Suite)을 선택한다. 이후 공유 비밀 생성을 위해 공개키와 비밀키 쌍을 무작위로 생성한다. 선택한 Cipher Suite과 공개키를 서버에 전송한다.

- Cipher Suite 선택. ex) AES-GCM, SPECK-CTR, ...
- A의 공개키/비밀키 쌍인 Pu_A, Pr_A 생성
- Cipher Suite, Pu_A 전달

2) ServerHello

서버는 클라이언트가 선택한 Cipher Suite이 사용 가능한지, 공개키가 유효한지 검사한다. 그 후, 공유 비밀 생성을 위해 공개키와 비밀키 쌍을 무작위로 생성하고, 해당 비밀키와 클라이언트의 공개키를 사용해 ECDH로 공유 비밀을 계산한다. 무작위로 선택한 salt를 사용하여 HKDF를 통해 대칭 키를 유도한 후, 인증서, 공개키 및 salt를 클라이언트에 전송한다.

- Cipher Suite 검증
- Pu_A 검증
- B의 공개키/비밀키 쌍인 Pu_B, Pr_B 생성

- ECDH에서 $Pu_A \cdot Pr_B$ 를 통해 공유 비밀 SharedSecret 생성
- salt 생성
- HKDF에서 salt, SharedSecret 을 통해 대칭 키 유도
- $Sig_B, Pu_A, salt$ 전달

3) ClientHelloDone

클라이언트는 서버가 보낸 인증서를 ECDSA를 통해 검증한 후, 서버의 공개키와 자신의 비밀키로부터 ECDH로 공유 비밀을 계산한다. 서버가 제공한 salt를 통해 HKDF로 대칭 키를 유도한다. 인증이 정상적으로 되었다면, 보안 통신을 시작할 준비가 되었다는 신호로 True를 서버에 전송한다.

- ECDSA를 통해 Sig_B 검증
- ECDH에서 $Pu_B \cdot Pr_A$ 를 통해 공유 비밀 SharedSecret 생성
- HKDF에서 salt, SharedSecret 을 통해 대칭 키 유도
- 서명 검증 결과(True) 전달

4) ServerHelloDone

서버는 클라이언트의 준비 신호를 확인한 후, 자신도 보안 통신을 시작할 준비가 되었음을 나타내는 True를 클라이언트에 전송한다. 이후 서버와 클라이언트는 앞서 설정한 Cipher Suite과 대칭 키를 사용하여 보안 통신을 시작한다.

클라이언트가 서버를 인증하는 방식은 CA(Certificate Authority)를 활용한 구조를 참고하였다. 그림 36은 클라이언트가 서버를 인증하는 과정을 나타낸다.

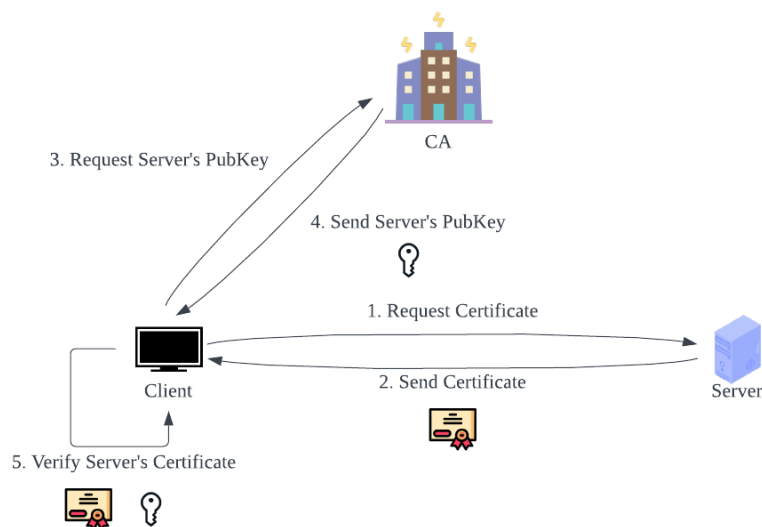


그림 36 - 클라이언트가 서버를 인증하는 과정

클라이언트는 ClientHelloDone 단계에서 서버 인증서를 검증한다. 이때 서버 인증서의 공개키는 CA가 가지고 있으며, 클라이언트는 CA에 서버 인증서의 공개키를 요청한다. 클라이언트는 1) 서버에게서 받은 인증서와 2) CA에서 받은 서버 인증서의 공개키를 가지고 ECDSA의 서명 검증 함수를 사용, 서버의 인증서를 검증한다. 해당 구조는 간소화되었지만, 클라이언트는 신뢰할 수 있는 제 3자인 CA를 통해 서버 인증서의 공개키를 안전하게 확보할 수 있다. 따라서, 간소화된 형태임에도 불구하고 충분한 보안성을 제공할 수 있다.

4.8. 게임 서버

게임 서버는 고성능의 비동기 소켓 I/O를 필요로 한다. IOCP 모델의 경우 Select 모델이나 WSAEventSelect 모델보다 성능이 뛰어나며[22], 스레드를 직접 관리하지 않기에 멀티스레드 프로그래밍에서 다른 모델에 비해 큰 이점을 얻을 수 있다. 특히 C#의 경우, .NET Framework에서 제공하는 SocketAsyncEventArgs 클래스를 활용하면 C++에 보다 쉽게 IOCP 서버를 구현할 수 있다. .NET Framework는 IOCP와 비동기 소켓 I/O를 위한 고수준의 API를 제공하며, 이는 .NET의 설계 취지인 생산성과 확장성에 맞춰져 있다.

비동기 소켓 I/O는 단순히 Send, Recv만을 처리하는 것이 아니라, Accept, Connect, Disconnect 작업도 포함된다. 일반적인 소켓 서버 처럼, IOCP 서버 또한 Bind, Listen, Accept의 과정이 필요하다. 그림 37은 일반적인 소켓 서버가 클라이언트의 연결을 받아들이는 Accept 과정을 나타낸다.

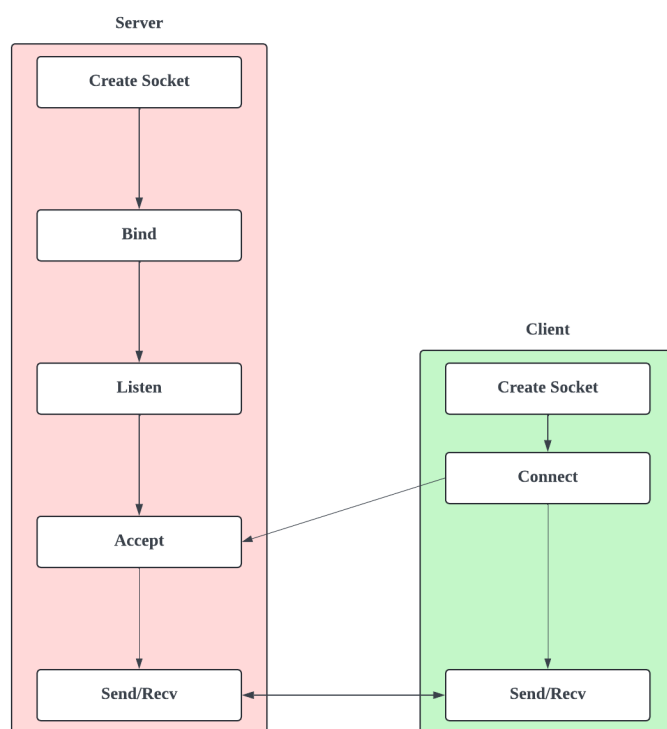


그림 37 - 서버가 클라이언트의 연결을 Accept 하는 과정

클라이언트가 서버에 Connect 요청을 보내면, 서버는 해당 연결을 Accept 하여 TCP 연결이 수립된다. IOCP 서버도 이 과정과 동일하지만, 모든 요청을 비동기로 처리한다는 점에서 차이가 있다. 그림 38과 그림 39는 IOCP 서버가 클라이언트의 연결을 비동기로 처리하는 과정을 나타낸다.

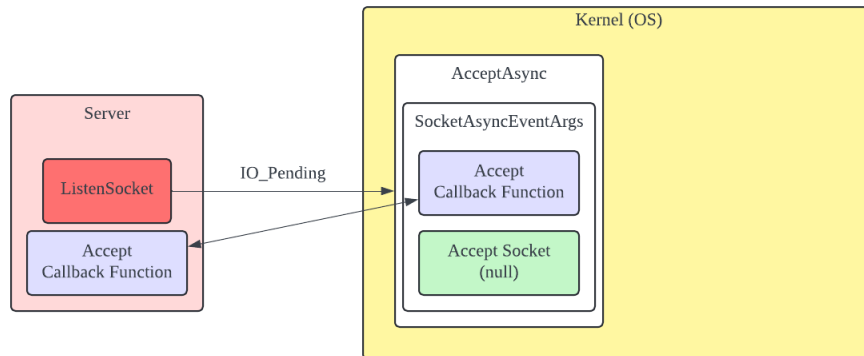


그림 38 - AcceptAsync 과정 - IO_Pending 상태

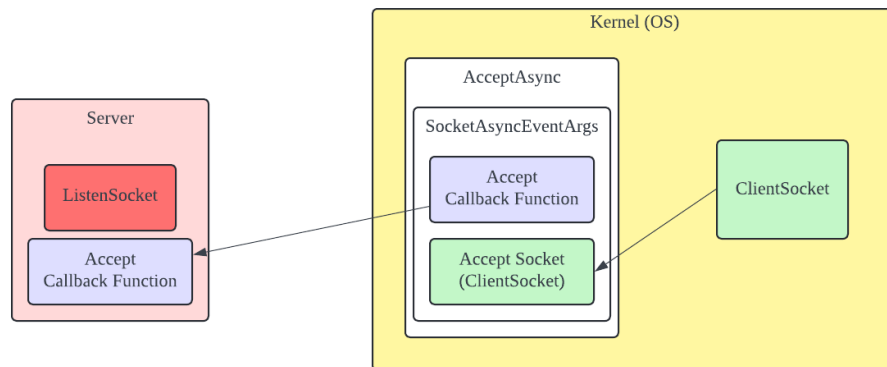
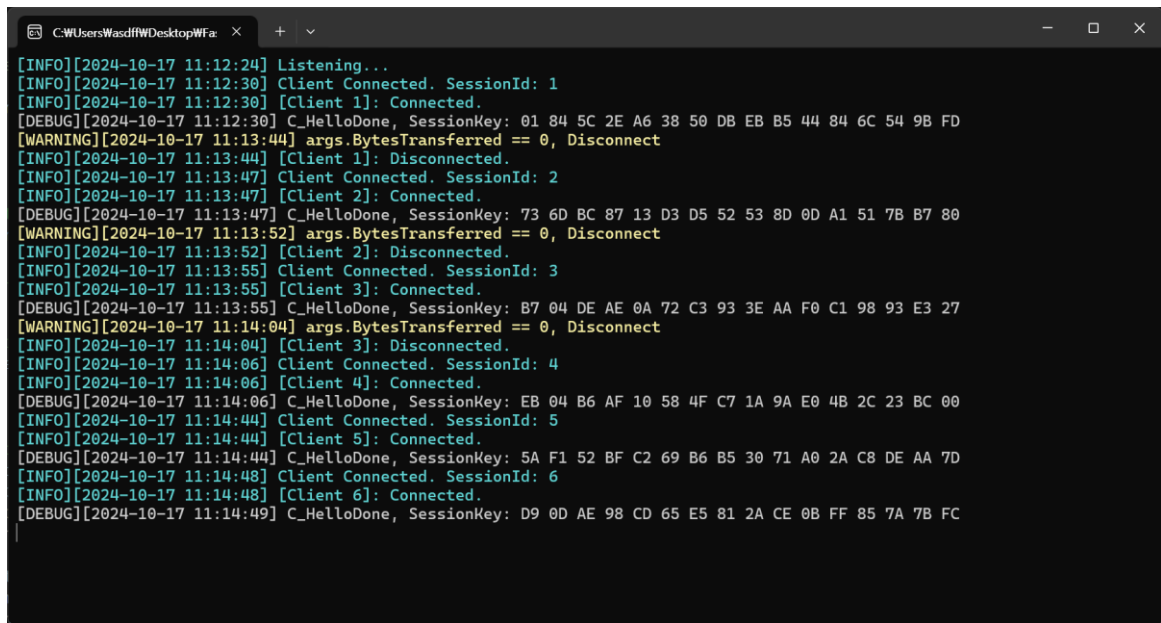


그림 39 - AcceptAsync 과정 - Accept 요청을 처리한 상태

서버는 Bind, Listen이 완료된 상태의 소켓에 대해 AcceptAsync 메서드를 호출한다. AcceptAsync는 SocketAsyncEventArgs 객체를 인자로 받으며, 이 객체에 Accept 완료 시 호출될 콜백 함수를 등록한다. 이후 이 함수는 대기 상태(IO_Pending)로 들어가고, 작업이 완료될 때까지 기다린다. 클라이언트가 연결 요청을 보내면, OS는 해당 요청을 처리한 후, 클라이언트의 연결 정보가 담긴 SocketAsyncEventArgs를 콜백 함수에 전달하여 서버에서 처리되도록 한다.

뿐만 아니라, SendAsync, ReceiveAsync, ConnectAsync 같은 다른 비동기 함수들도 동일한 방식으로 동작한다. 이 비동기 I/O 함수들의 핵심은 논블로킹 방식으로 동작한다는 점이며, IO_Pending 상태일 경우 true를 반환하여 프로그램의 다른 작업이 중단되지 않고 진행되도록 한다. 따라서, 프로그래머는 스레드를 직접 관리할 필요가 없으며, 비동기 함수의 처리 또한 OS에 의해 자동으로 관리된다.



```
[INFO][2024-10-17 11:12:24] Listening...
[INFO][2024-10-17 11:12:30] Client Connected. SessionId: 1
[INFO][2024-10-17 11:12:30] [Client 1]: Connected.
[DEBUG][2024-10-17 11:12:30] C_HelloDone, SessionKey: 01 84 5C 2E A6 38 50 DB EB B5 44 84 6C 54 9B FD
[WARNING][2024-10-17 11:13:44] args.BytesTransferred == 0, Disconnect
[INFO][2024-10-17 11:13:44] [Client 1]: Disconnected.
[INFO][2024-10-17 11:13:47] Client Connected. SessionId: 2
[INFO][2024-10-17 11:13:47] [Client 2]: Connected.
[DEBUG][2024-10-17 11:13:47] C_HelloDone, SessionKey: 73 6D BC 87 13 D3 D5 52 53 8D 0D A1 51 7B B7 80
[WARNING][2024-10-17 11:13:52] args.BytesTransferred == 0, Disconnect
[INFO][2024-10-17 11:13:52] [Client 2]: Disconnected.
[INFO][2024-10-17 11:13:55] Client Connected. SessionId: 3
[INFO][2024-10-17 11:13:55] [Client 3]: Connected.
[DEBUG][2024-10-17 11:13:55] C_HelloDone, SessionKey: B7 04 DE AE 0A 72 C3 93 3E AA F0 C1 98 93 E3 27
[WARNING][2024-10-17 11:14:04] args.BytesTransferred == 0, Disconnect
[INFO][2024-10-17 11:14:04] [Client 3]: Disconnected.
[INFO][2024-10-17 11:14:06] Client Connected. SessionId: 4
[INFO][2024-10-17 11:14:06] [Client 4]: Connected.
[DEBUG][2024-10-17 11:14:06] C_HelloDone, SessionKey: EB 04 B6 AF 10 58 4F C7 1A 9A E0 4B 2C 23 BC 00
[INFO][2024-10-17 11:14:44] Client Connected. SessionId: 5
[INFO][2024-10-17 11:14:44] [Client 5]: Connected.
[DEBUG][2024-10-17 11:14:44] C_HelloDone, SessionKey: 5A F1 52 BF C2 69 B6 B5 30 71 A0 2A C8 DE AA 7D
[INFO][2024-10-17 11:14:48] Client Connected. SessionId: 6
[INFO][2024-10-17 11:14:48] [Client 6]: Connected.
[DEBUG][2024-10-17 11:14:49] C_HelloDone, SessionKey: D9 0D AE 98 CD 65 E5 81 2A CE 0B FF 85 7A 7B FC
```

그림 42 - 서버 실행 화면

4.9. 게임 클라이언트

게임 클라이언트는 플레이어의 입력 처리, 패킷 처리를 통한 GameObject 관리, UI 구현 등의 기능을 담당하지만, 세부적인 구현 내용은 본 연구의 주제를 벗어나므로 본문에서 제외하였다. 이번 절에서는 메인 스레드에서 패킷이 처리되게 하는 구조에 대해 설명하고자 한다.

먼저, 게임 클라이언트도 게임 서버와 마찬가지로 비동기 소켓 I/O를 통해 패킷의 송수신을 처리한다. 해당 과정은 그림 38, 39와 같으며, OS가 자체적으로 스레드를 관리하여 I/O를 처리하고 콜백 함수를 호출한다. 이 때 OS가 관리하는 스레드는 유니티의 메인 루프가 동작하는 스레드가 아니며, 게임의 특성상 패킷을 처리할 때 핸들러가 GameObject를 직접 접근하는 경우가 생긴다. 따라서, 게임 서버와는 다르게 패킷을 처리하는 핸들러가 메인 스레드에서 동작할 수 있도록 추가적인 작업이 필요하다.

본 연구에서는 이러한 문제를 해결하기 위해 이벤트 큐(Event Queue) 패턴을 사용하였다. 이벤트 큐 패턴은 이벤트를 받는 시점과 이벤트를 처리하는 시점을 분리하는 패턴이다. 즉, 패킷을 받는 시점과 패킷을 처리하는 시점을 나누는 방식이다. 패킷을 받는 시점은 비동기 I/O를 통해 결정되지만, 패킷 처리 시점은 메인 스레드에 접근할 수 있는 MonoBehaviour를 상속받는 객체에서 결정한다. 따라서 본 연구에서는 PacketQueue 클래스와, NetworkManager 클래스를 추가하였다. 그림 43은 패킷을 받고 처리하는 과정을 나타낸다.

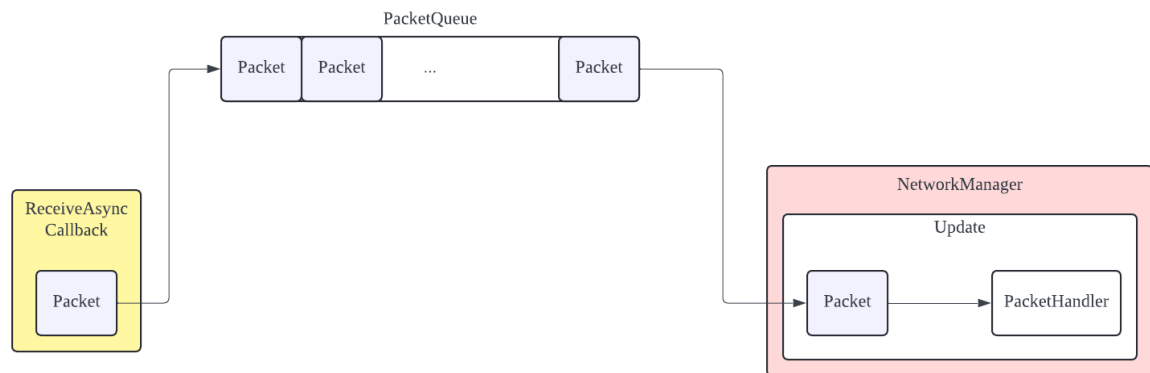


그림 43- 패킷을 받고 처리하는 과정

NetworkManager는 MonoBehaviour를 상속받는 객체로, Update 함수를 매 프레임마다 메인 스레드에서 실행한다. 따라서, PacketQueue를 전역 객체로 사용하고, ReceiveAsync를 통해 받은 패킷을 PacketQueue에 넣은 뒤, NetworkManager가 해당 패킷을 꺼내 적절한 핸들러를 호출하면 문제를 해결할 수 있다. PacketQueue의 경우 여러 스레드가 동시에 접근할 수 있기에 (Critical Section), Push, Pop을 할 때 lock을 걸어 race condition을 방지하였다. 그림 44는 게임 클라이언트 작성 환경과 실행 화면이다.

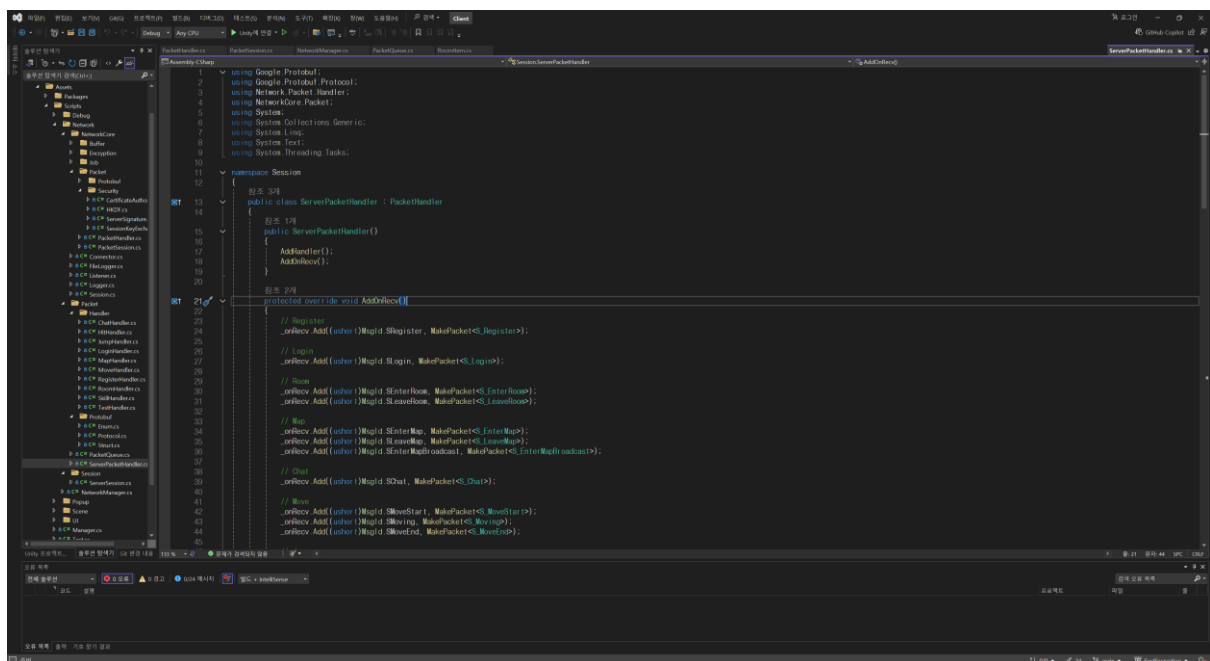


그림 44 - 클라이언트 코드 일부와 작성 환경

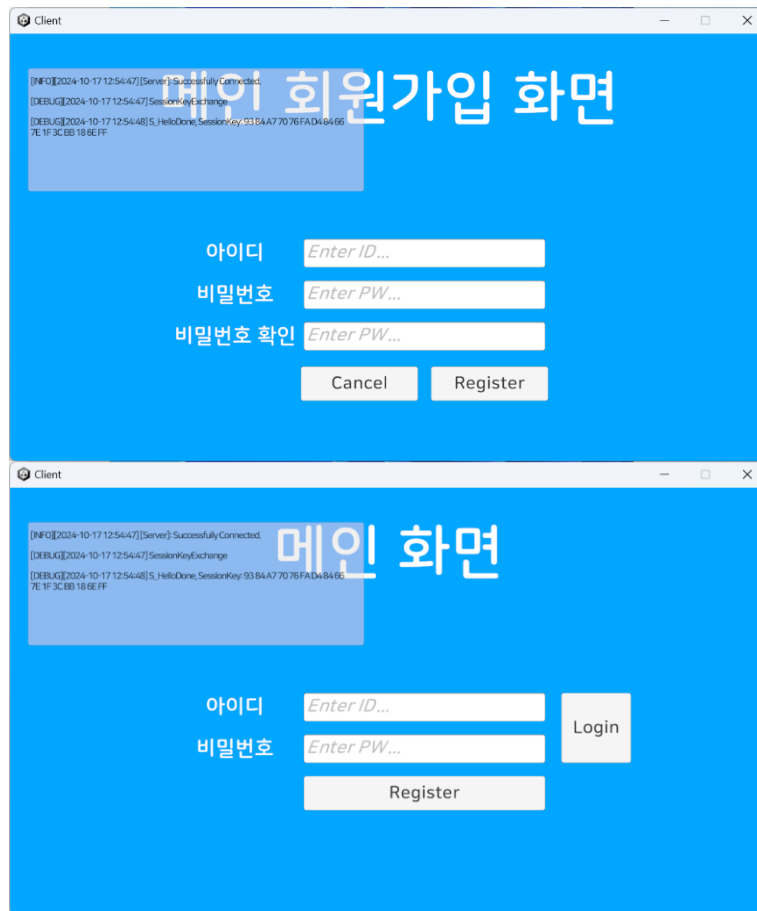


그림 45 - 로그인 및 회원가입 화면

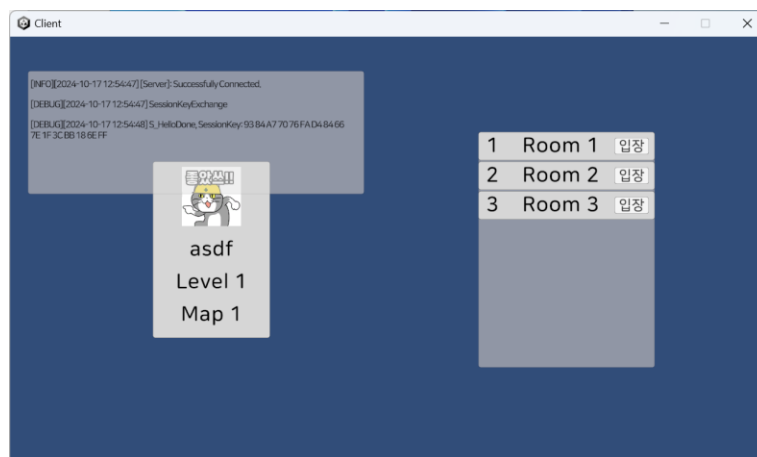


그림 46 - 방 입장 화면, 서버로부터 플레이어의 정보와 방 정보를 받아온다.

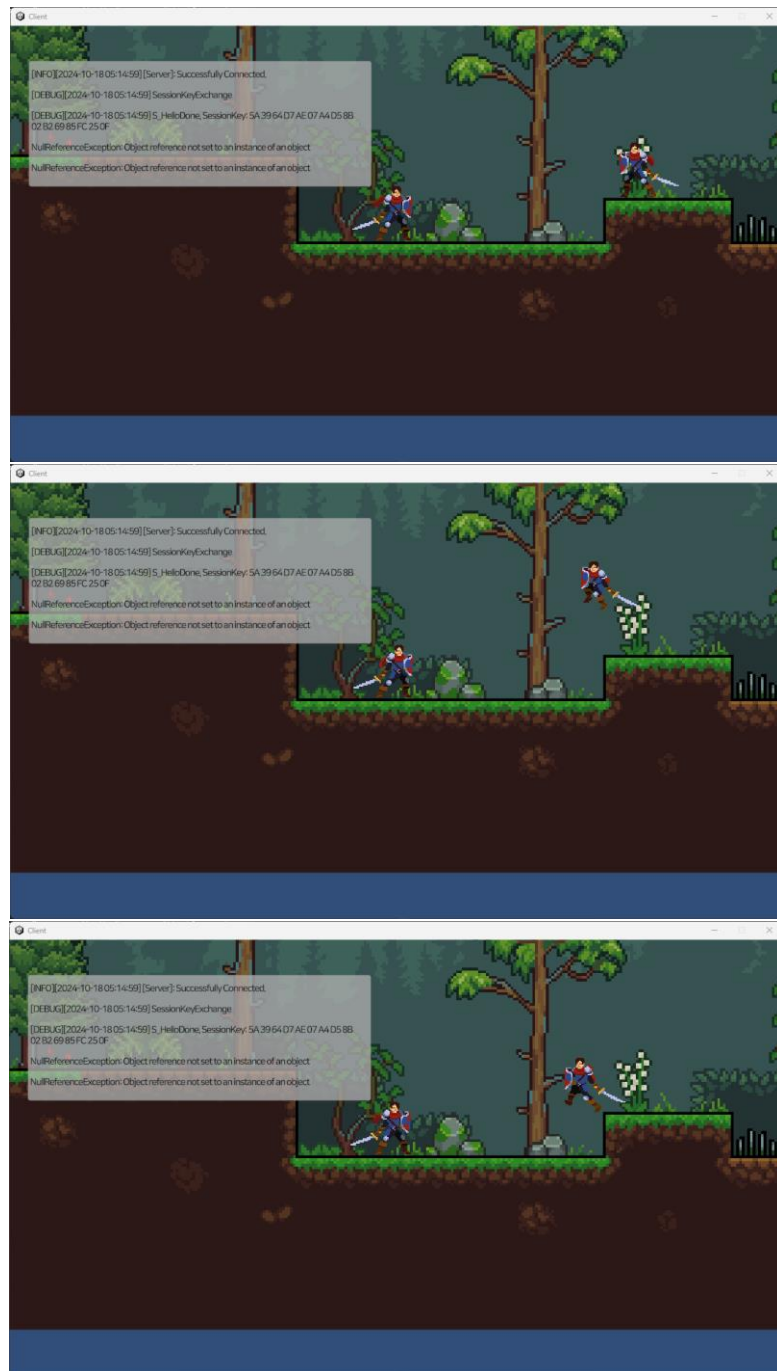


그림 47 - 두 플레이어의 위치 정보가 점프, 이동을 하며 동기화된 모습이다.

5. 시뮬레이션 및 적용

5.1. 안전한 게임 통신 환경 시뮬레이션

앞서 구현한 여러 암호 알고리즘들을 안전한 게임 통신 환경을 위해 적용해보았다. 그림 48, 49는 각각 서버와 클라이언트에서 세션을 수립할 때 진행되는 Handshake Protocol 과정의 로

그를 보여준다.

```
C:\Users\Administrator\Desktop\FastEncryption\Server\Server\bin\Debug\net8.0\Server.exe

[DEBUG][2024-10-18 05:39:54] Recv C_Hello_Done
[DEBUG][2024-10-18 05:39:54] Set Operation Mode: NetworkCore.Encryption.BlockCipher.OperationMode.ECB
[DEBUG][2024-10-18 05:39:54] Derived Session Key: 40 6E FF 34 D5 3B B0 23 B7 42 7B 84 49 A6 E8 30
[DEBUG][2024-10-18 05:39:54] Secure Communication Established
[DEBUG][2024-10-18 05:39:54] C_HelloDone, SessionKey: 40 6E FF 34 D5 3B B0 23 B7 42 7B 84 49 A6 E8 30
[DEBUG][2024-10-18 05:39:54] Sent S_Hello_Done Packet: Success=true
[INFO][2024-10-18 05:39:59] Client Connected. SessionId: 3
[INFO][2024-10-18 05:39:59] [Client 3]: Connected.

[DEBUG][2024-10-18 05:39:59] Recv C_Hello
[DEBUG][2024-10-18 05:39:59] Cipher Suite Validation: True, Cipher Suite: AesCbc
[DEBUG][2024-10-18 05:39:59] Parsed Client Public Key: X=605983694682563556809457534908676195346952213300305823277652862
38724459577390, Y=98358503390741344300748048854536959938596287551784073742579220787253721367502
[DEBUG][2024-10-18 05:39:59] Client Public Key Validation: True
[DEBUG][2024-10-18 05:39:59] Generated Server Public Key: X=880155936313109244667053839781293111281200430568576281359910
24614701229609385, Y=82621904966513878976479945719869834410893473973857707101783778158364996721824
[DEBUG][2024-10-18 05:39:59] Calculated Shared Secret: A2 3D 4F 2A 44 99 03 9A 02 57 E2 37 8C 27 5A CE 55 AC E4 77 CE 11
86 0E EF 8E 1B 4C D0 38 FC 2E
[DEBUG][2024-10-18 05:39:59] Server Signature: E6 7A 97 17 CC F9 68 41 48 9D 65 41 F4 F6 AD B1 2D 17 B5 9A 6B EF 84 7B 6
1 83 B8 FC F1 6A 32 EB 9A E6 BA 6D 63 77 06 84 9A 6A 9F C3 88 CF 02 32 D8 5C 26 EA 0D 1F E7 43 7A DB 48 DE 58 36 43 33
[DEBUG][2024-10-18 05:39:59] Generated Salt: 27 36 0F 04 F3 F6 7B AA B1 48 72 FD C2 A2 0E BE
[DEBUG][2024-10-18 05:39:59] Sent S_Hello Packet: Success=True

[DEBUG][2024-10-18 05:40:00] Recv C_Hello_Done
[DEBUG][2024-10-18 05:40:00] Set Operation Mode: NetworkCore.Encryption.BlockCipher.OperationMode.CBC
[DEBUG][2024-10-18 05:40:00] Derived Session Key: 45 86 57 D6 64 A7 DE C5 04 75 DE 03 69 8C 9B D5
[DEBUG][2024-10-18 05:40:00] Secure Communication Established
[DEBUG][2024-10-18 05:40:00] C_HelloDone, SessionKey: 45 86 57 D6 64 A7 DE C5 04 75 DE 03 69 8C 9B D5
[DEBUG][2024-10-18 05:40:00] Sent S_Hello_Done Packet: Success=true
```

그림 48 - Handshake Protocol 서버 로그

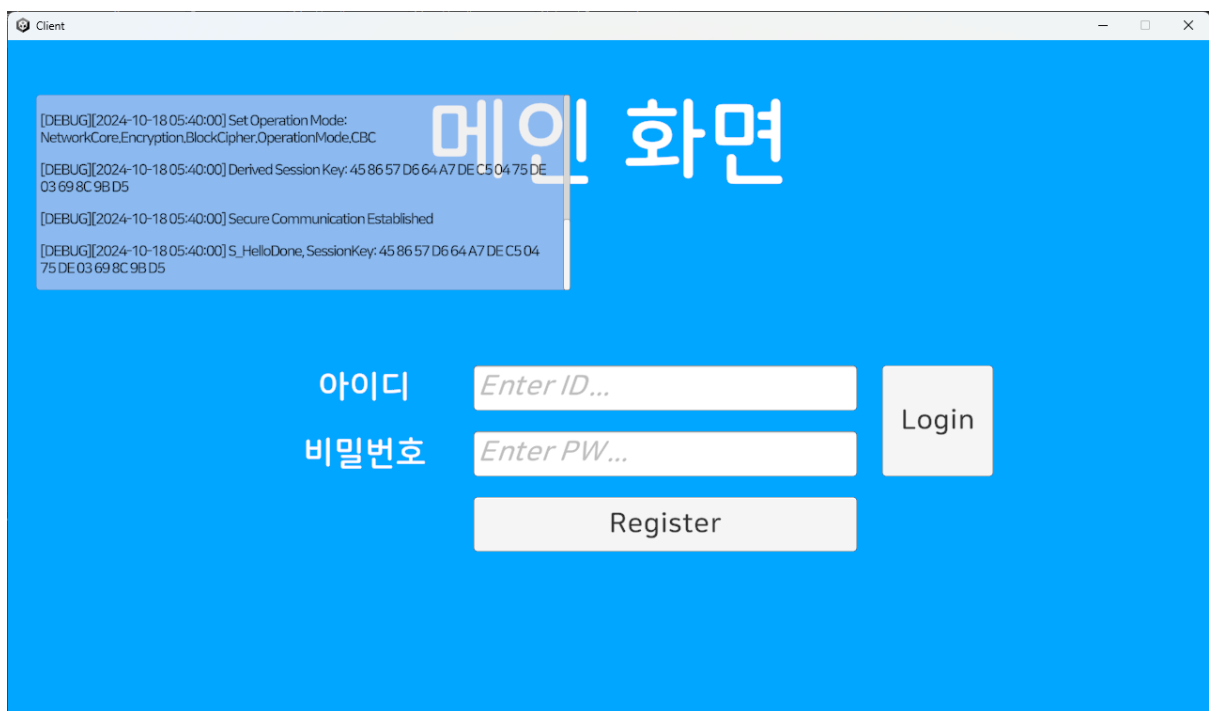


그림 49 - Handshake Protocol 클라이언트 로그

이 과정을 통해 클라이언트는 서버의 인증을 마치고, 보안 통신을 위한 대칭 키 유도까지 성공적으로 완료했다. 그림 50은 클라이언트가 암호화된 패킷을 송신하고, 수신한 패킷을 복호화한 모습을 나타낸다.

```

C:\Users\Administrator\Desktop\FastEncryption\Server\DummyClient\bin\Debug\net8.0\DummyClient.exe

[DEBUG][2024-10-18 05:52:42] Recv S_Hello_Done
[DEBUG][2024-10-18 05:52:42] Set Operation Mode: NetworkCore.Encryption.BlockCipher.OperationMode.ECB
[DEBUG][2024-10-18 05:52:42] Derived Session Key: FF 8E 79 41 6D 97 B5 CC CF 8D 34 AF 57 3C 45 8D
[DEBUG][2024-10-18 05:52:42] Secure Communication Established
[DEBUG][2024-10-18 05:52:42] S_HelloDone, SessionKey: FF 8E 79 41 6D 97 B5 CC CF 8D 34 AF 57 3C 45 8D
[INFO][2024-10-18 05:52:43] Client_1: DaDtFhx8gJ6GmoBvZHJ433fKQ4sm6oRTH5Mw86Tt5uOMmbbcjw
[DEBUG][2024-10-18 05:52:43] Encrypted Packet : 8D 3A 96 1C 75 1B 16 AC 1C 4D F2 D1 92 39 F1 11 40 DD D5 31 02 C6 79 94
4B 69 A1 65 AE 66 EB 6D BE 78 C4 70 46 F9 11 A7 08 AE 46 95 08 6D C6 A8 7F 5E 21 66 73 F3 EE 4F 69 A3 21 67 61 47 75 14
[DEBUG][2024-10-18 05:52:43] Decrypted Packet : 0A 32 44 61 44 74 46 48 78 30 67 4A 36 47 6D 6F 42 76 5A 48 4A 34 33 33
66 4B 51 34 73 6D 36 6F 52 54 48 35 4D 77 38 36 54 74 35 75 4F 4D 62 6D 62 63 6A 77
[INFO][2024-10-18 05:52:43] [Server]: DaDtFhx8gJ6GmoBvZHJ433fKQ4sm6oRTH5Mw86Tt5uOMmbbcjw
[INFO][2024-10-18 05:52:44] Client_1: uDGjQwB885t70c70sKaWkx4yg6dKvTtM9dUqrHTzN6L7emSqlpB
[DEBUG][2024-10-18 05:52:44] Encrypted Packet : 9B 4F D5 37 82 87 3E F6 65 A1 2A 19 69 BA F1 F6 C3 42 14 E2 6D D5 08 5C
36 03 C8 E7 32 3F 75 43 55 21 9B 01 38 B3 F1 DB 5A CC D6 BB 2A A7 AE C8 A6 20 0F C3 23 84 0B BC CB 27 88 DC 64 0B 9B 59
[DEBUG][2024-10-18 05:52:44] Decrypted Packet : 0A 32 75 44 47 6A 51 77 42 38 38 35 74 37 4F 63 37 30 73 4B 61 57 68 78
34 79 67 36 64 4B 56 74 4D 39 64 55 71 72 48 54 7A 4E 36 4C 37 65 6D 53 71 6C 70 42
[INFO][2024-10-18 05:52:44] [Server]: uDGjQwB885t70c70sKaWkx4yg6dKvTtM9dUqrHTzN6L7emSqlpB
[INFO][2024-10-18 05:52:44] Client_1: iW8gyYYY3UKy66mfc5SSSftZab4DP0QRzvhzuEI9OS1Iw0RsIR
[DEBUG][2024-10-18 05:52:44] Encrypted Packet : 1B C0 09 AB 89 00 80 FE 1B 75 59 7B 18 6E 7D 79 BF 94 F4 A4 B3 77 22 58
94 6A 93 B3 73 E5 E1 53 22 AA 72 72 BC E8 A2 2E D8 A6 41 65 7C 45 55 13 C4 B9 ED 7B 71 90 59 1A F7 0E B5 3C 10 02 0F C6
[DEBUG][2024-10-18 05:52:44] Decrypted Packet : 0A 32 69 57 38 67 79 59 59 59 53 55 4B 79 36 36 6D 66 63 35 53 53 53 66
74 5A 61 62 34 44 50 30 51 52 7A 76 68 7A 75 45 49 39 4F 53 31 49 77 30 52 73 49 52
[INFO][2024-10-18 05:52:44] [Server]: iW8gyYYY3UKy66mfc5SSSftZab4DP0QRzvhzuEI9OS1Iw0RsIR
[INFO][2024-10-18 05:52:45] Client_1: xRDkK9U1zV46F7ZVnLWxJAah4m8rRiBhnhX6eVz580D8Wsqw9
[DEBUG][2024-10-18 05:52:45] Encrypted Packet : 9B 52 3D 2C A9 B5 E4 38 3F CC 02 7F 25 46 45 20 B0 3F 89 2C 85 CE 01 0C
D7 0C 2A AB FA 38 94 87 71 B9 7C F6 18 2B 6F 25 58 40 41 F3 1B CF 0B A7 15 CF D8 46 81 34 F7 97 C9 54 B8 38 A4 D4 46 04
[DEBUG][2024-10-18 05:52:45] Decrypted Packet : 0A 32 78 52 44 4B 6B 39 55 31 7A 56 34 36 46 37 5A 56 6E 4C 57 78 4A 41
61 68 34 6D 38 72 52 69 42 68 68 6E 58 36 65 56 7A 35 38 4F 44 38 57 73 77 71 77 39
[INFO][2024-10-18 05:52:45] [Server]: xRDkK9U1zV46F7ZVnLWxJAah4m8rRiBhnhX6eVz580D8Wsqw9
[INFO][2024-10-18 05:52:45] Client_1: IZiPuh9JMFREtX8G2QF7hpDRLD2HtmKeUDYo8DRLxYurnidw

```

그림 50 - 클라이언트의 패킷 암호화 및 복호화

위 그림에서 클라이언트는 `iW8gyYYY3UKy66mfc5SSSftZab4DP0QRzvhzuEI9OS1Iw0RsIR`를 암호화하여 서버에 전송한다. 이후 서버에 자신이 보낸 문자열을 다시 받게 된다(echo). 이는 단순한 랜덤 문자열이며, 길이는 50 Bytes 이다. 이때 암호화된 패킷은 `1B C0 ... C6`이며, 복호화된 패킷은 `0A 32 ... 52` 이다. 복호화된 패킷의 `0A`는 패킷 아이디를 의미하며, `32`는 헤더 사이즈가 포함된 패킷 길이를 의미한다. `69, 57, ..., 52`는 클라이언트가 전송한 랜덤 문자열의 아스키 코드이다. 따라서, 성공적으로 대칭 키를 이용한 암호화 및 복호화가 이루어짐을 알 수 있다.

5.2. 블록 암호 알고리즘별 성능 측정

각 블록 암호 알고리즘의 성능을 비교하기 위해, 패킷을 암호화 및 복호화할 때 걸리는 시간을 측정하였다. 측정은 패킷을 암호화 또는 복호화하기 직전에 시작하며, 패킷이 암호화 또는 복호화된 직후에 종료된다. 아래 코드는 암호화 시간을 측정하는 코드의 일부이다.

```

public void Send(IMessage packet)
{
    ...

    Stopwatch stopwatch = Stopwatch.StartNew();
    byte[] encryptionPacket = OperationMode.Encrypt(packet.ToArray());
    stopwatch.Stop();
    FileLogger.Instance.Log(new CipherLogObject
    {
        IsEncrypt = true,
        AlgorithmName = OperationMode.AlgorithmName,
        OperationModeName = OperationMode.ModeName,
        ElapsedMilliseconds = (double)stopwatch.ElapsedTicks / Stopwatch.Frequency * 1000,
    });
    ...

    Send(new ArraySegment<byte>(sendBuffer));
}

```

시간 측정을 위해 .NET 에서 제공하는 고성능 시간 측정 라이브러리인 Stopwatch를 사용하였다. Stopwatch.StartNew()를 사용해 시간 측정을 시작하고, Stopwatch.Stop()을 호출해 측정을 종료한다. 측정된 시간은 밀리초(ms) 단위로 기록된다. 복호화 과정도 암호화와 동일한 방식으로 진행된다.

실제 유니티 클라이언트에서 성능을 측정한 자료를 사용할 수 있지만, 테스트의 편의성을 위해 콘솔에서 동작하는 더미 클라이언트(DummyClient)를 작성하였다. 더미 클라이언트는 단순히 서버와 연결하여 패킷을 에코(echo)하도록 지시하는 프로그램이다. 그림 51은 유니티 클라이언트와 더미 클라이언트가 TWINE-CFB를 사용할 때의 암호화와 복호화에 걸리는 시간을 그래프로 나타낸 것이다.

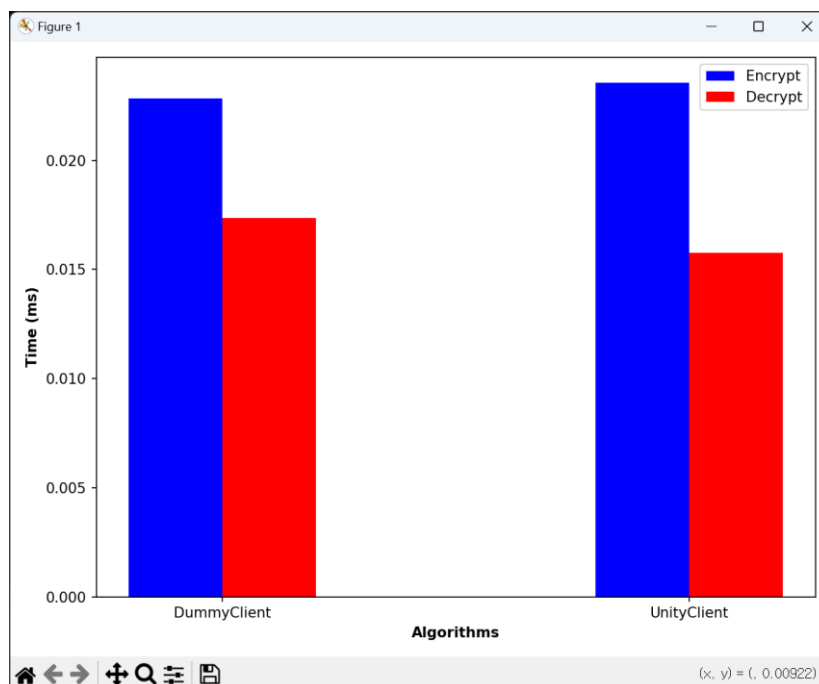


그림 51 - 더미 클라이언트와 유니티 클라이언트의 TWINE-CFB 성능 비교 그래프

위 그래프를 통해 더미 클라이언트와 유니티 클라이언트 간의 성능 차이가 거의 없음을 확인할 수 있다. 따라서 본 연구에서는 각 블록 암호 알고리즘의 성능을 비교할 때, 더미 클라이언트를 사용할 것이다.

성능 측정 시 유의할 점은, 일반적인 블록 암호 알고리즘의 성능 측정과, 유니티 클라이언트에서의 성능 측정 간에 차이가 있을 수 있다는 것이다. 이는 운영체제의 스레드 스케줄링과 관련이 있다. 운영체제는 인터럽트가 발생하면 현재 실행 중인 스레드를 멈추고 컨텍스트 스위치를 수행한 뒤, 인터럽트 처리가 끝나면 멈췄던 스레드를 다시 복원한다. 그림 52는 더미 클라이언트에서 AES-GCM을 사용했을 때, Thread.Sleep()을 호출한 경우와 호출하지 않은 경우의 성능 차이를 보여준다.

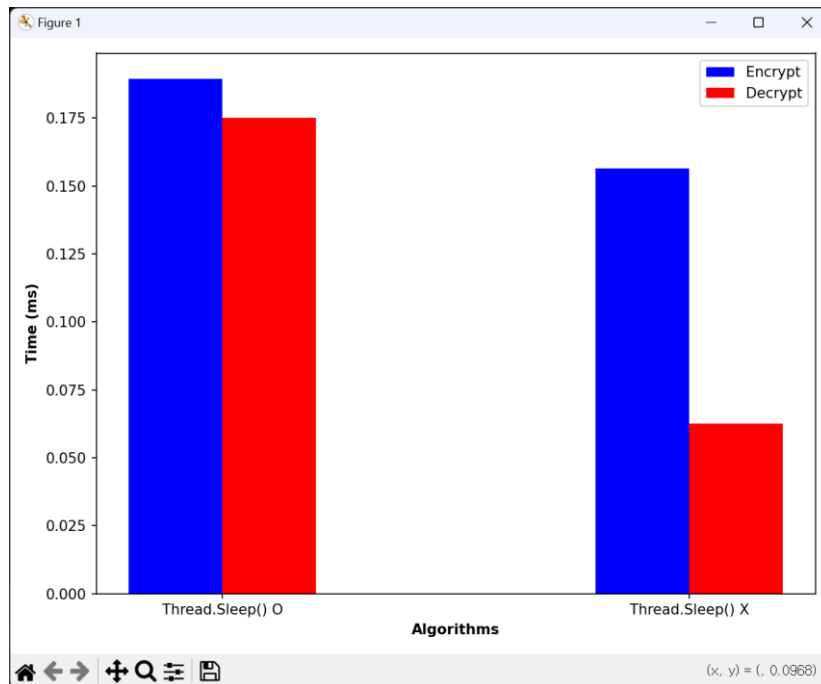


그림 52 - AES-GCM 사용 시 더미 클라이언트의 인터럽트 발생 유무에 따른 성능 차이

Thread.Sleep()을 사용한 경우가 사용하지 않은 경우보다 성능이 떨어지는 것을 확인할 수 있다. 하지만, 실제 게임 환경에서는 I/O 인터럽트, 타이머 인터럽트 등 여러 종류의 인터럽트가 자주 발생한다. 따라서 Thread.Sleep()을 사용한 쪽이 현실적인 상황에 더 가깝다고 볼 수 있다. 본 연구에서는 이러한 현실적인 상황을 반영하여, Thread.Sleep()을 사용해 블록 암호 알고리즘들의 성능을 측정하였다.

GCM 모드는 다른 운영 모드와 달리 IV 외에도 AAD를 사용할 수 있으며, 암호화 과정에 태그 인증이 포함된다. 이는 GCM 모드의 특성으로, 태그 인증 과정을 생략하지 않고 실험을 진행했다. 또한, 실험의 공정성을 위해 AAD를 사용하지 않았다.

그림 53, 54, 55, 56, 57은 사용한 GCM을 제외한 운영모드에 각 블록 암호 알고리즘을 사용했을 때 성능 측정 결과를 나타낸다. 그림 58은 각 AES, ARIA, SPECK 에 대해 GCM 모드를 적용했을 때의 성능을 나타낸다

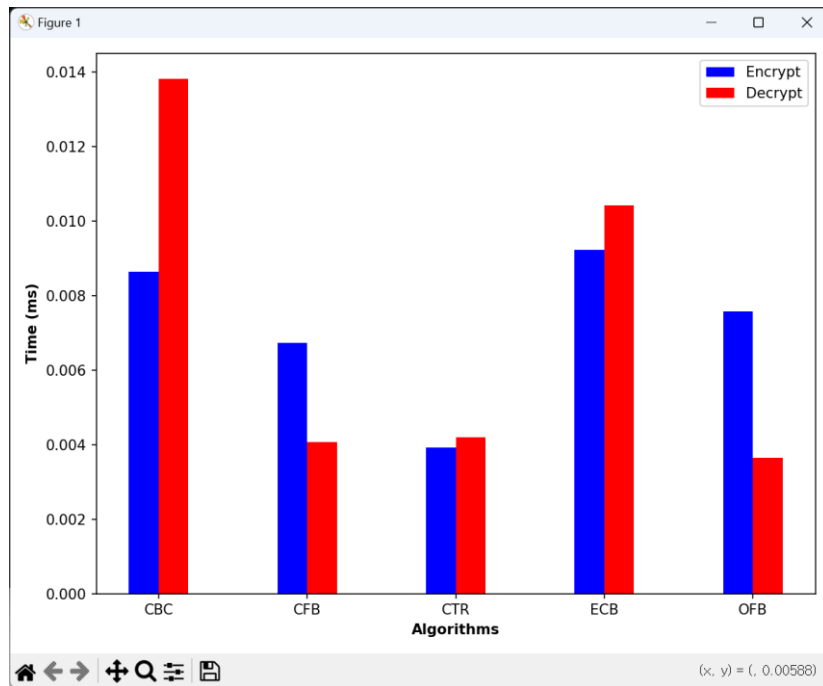


그림 53 - AES를 사용한 각 운영모드별 성능

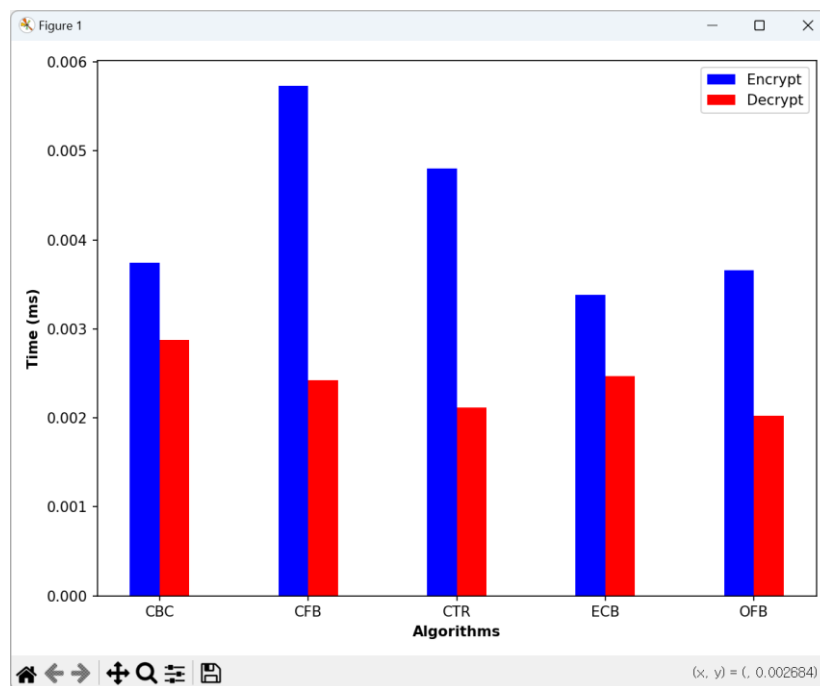


그림 54 - ARIA를 사용한 각 운영모드별 성능

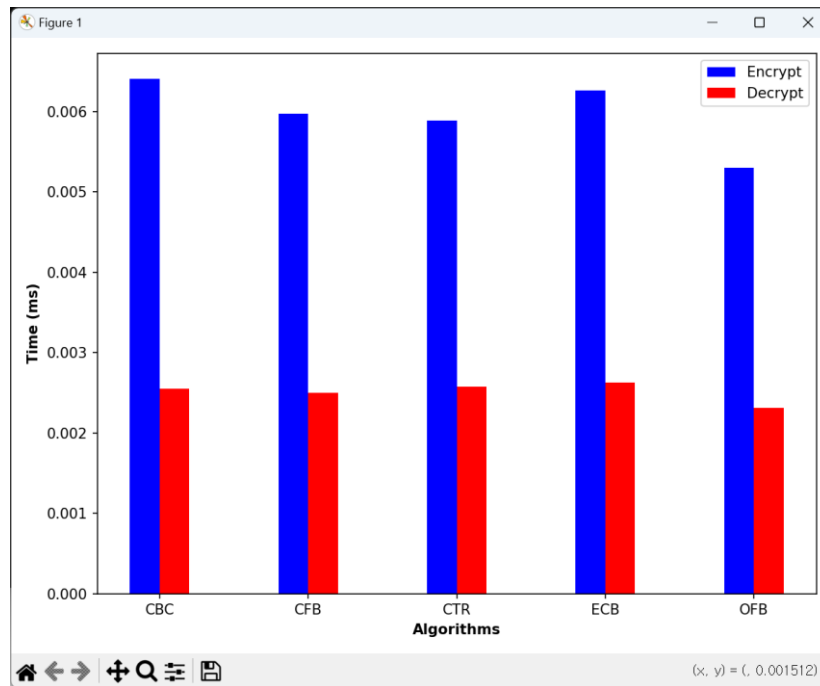


그림 55 - HIGHT를 사용한 각 운영모드별 성능

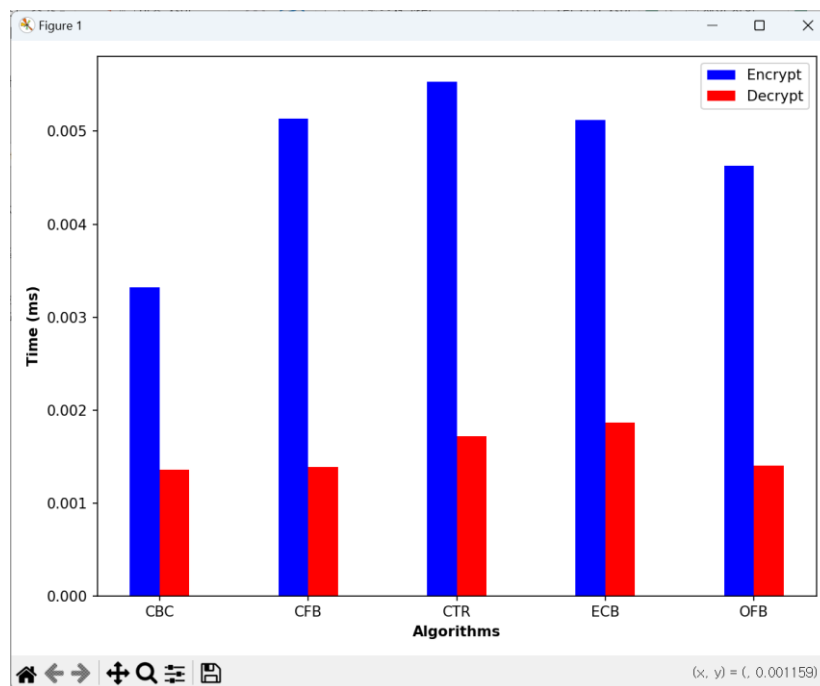


그림 56 - SPECK을 사용한 각 운영모드별 성능

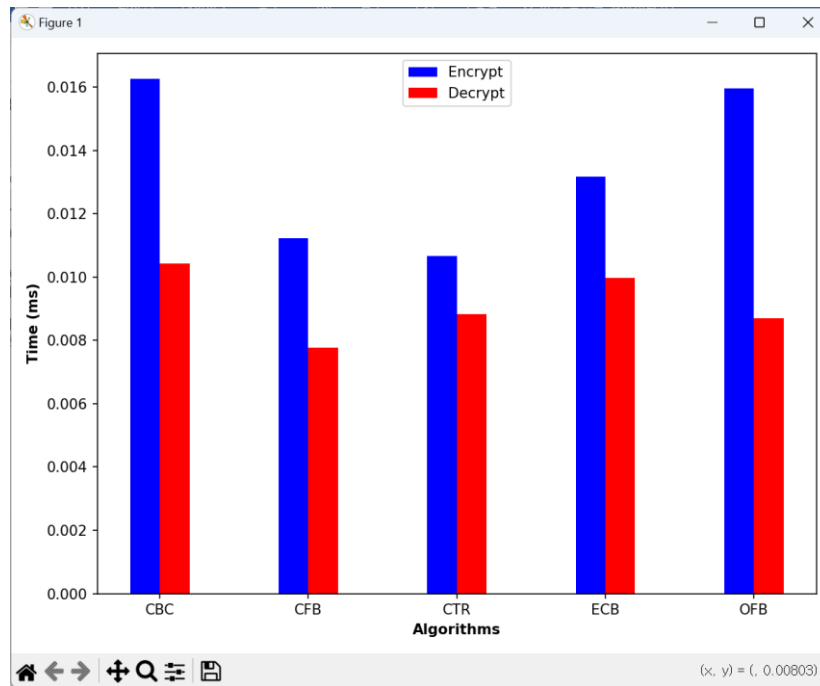


그림 57 - TWINE을 사용한 각 운영모드별 성능

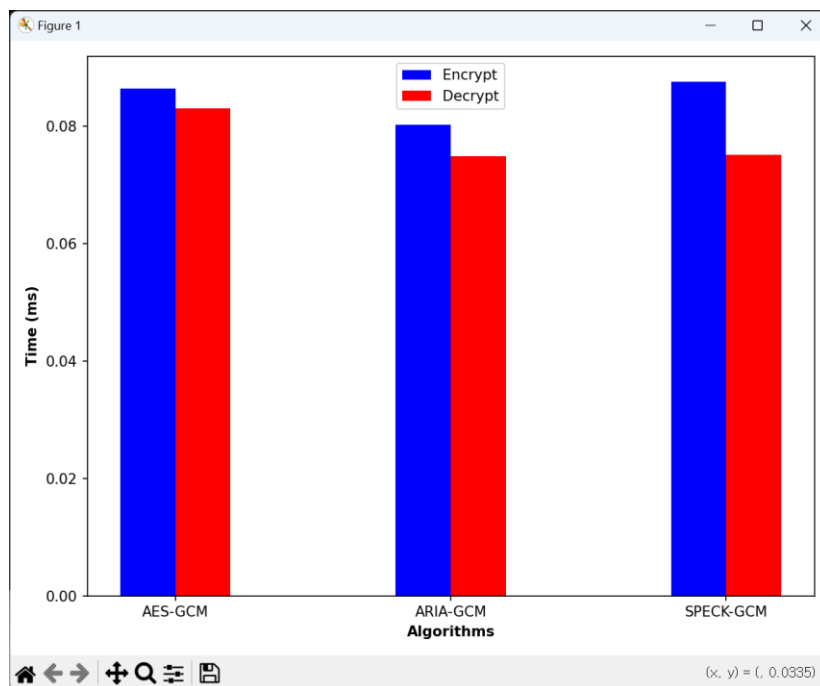


그림 58 - AES, ARIA, SPECK을 사용한 GCM 모드의 성능

6. 연구 결과

6.1. 분석 및 평가

본 연구에서 설계한 Handshake Protocol은 그림 49, 50을 통해 알 수 있듯이 정상적으로 작동했다. 서버 인증, 키 교환 및 공유 비밀 생성, 공유 비밀을 통한 대칭 키 유도, 대칭 키를 통한 패킷 암호화 및 복호화가 모두 성공적으로 이루어졌다. 이를 통해, 유니티 멀티플레이어 환경에서 안전한 통신 환경을 구축하는 데 성공하였음을 확인할 수 있다.

블록 암호 알고리즘의 성능을 비교한 결과, GCM을 제외한 다른 운영모드에서 SPECK이 가장 우수한 성능을 보였고, TWINE이 가장 저조한 성능을 보였다. 특히 SPECK은 경량 암호 알고리즘으로, 모바일 게임과 같은 자원이 제한된 환경에서 매우 유용할 수 있다. 반면, TWINE은 성능이 낮았지만, 이는 보안 강도를 고려한 설계 때문일 수 있다. HIGHT 알고리즘은 모든 운영 모드에서 비교적 일관된 성능을 보여, 특정 환경에서 안정적인 선택이 될 수 있음을 시사한다.

운영모드에 따른 성능 차이도 흥미로운 결과를 보여주었다. 각 운영모드는 암호화 과정에서의 복잡성, 패딩 방식, 인증 요구 사항 등에 영향을 받았다. 그림 58에서 알 수 있듯이, GCM 모드는 다른 운영모드에 비해 시간이 많이 소요되었다. 이는 GCM 모드가 인증 태그를 생성하고 인증하는 추가 과정이 있어, 해당 과정이 성능에 영향을 미친다는 것을 의미한다. 이러한 인증 태그는 무결성 및 인증성을 위해 필수적이지만, 성능 면에서는 부정적인 영향이 발생함을 알 수 있었다.

그러나 어떤 블록 암호 알고리즘과 운영 모드를 사용하더라도 성능 측정 결과는 모두 0.1 ms를 넘지 않았다. 이는 네트워크 환경에서의 RTT(Round Trip Time)에 비하면 매우 짧은 시간이다. 따라서, 본 연구에서 사용한 블록 암호 알고리즘들은 네트워크 통신에 실질적으로 영향을 미치지 않으며, 게임 환경에서 충분히 실용적으로 활용될 수 있음을 시사한다.

또한 본 연구는 Thread.Sleep()을 사용한 성능 측정 방식을 통해, 실제 게임 환경에서 자주 발생하는 I/O 인터럽트나 타이머 인터럽트와 같은 다양한 시스템 이벤트의 영향을 반영하였다. 이를 통해, 단순한 성능 비교가 아닌, 현실적인 성능 평가를 제공하였으며, 이는 암호화 알고리즘 선택 시 성능 뿐만 아니라 실제 환경에서의 안정성까지 고려할 수 있게 해준다. 특히, GCM 모드처럼 보안이 강화된 인증 태그를 생성하는 방식은 성능에 영향을 주지만, 더 높은 보안을 보장한다는 점에서 중요하다. 따라서 이러한 성능과 보안의 균형을 고려하는 것이 게임 개발이나 서버-클라이언트 환경의 암호화 설계에서 중요한 요소가 될 것이다.

6.2. 연구의 한계 및 추가 연구 제안

본 연구에서는 다양한 블록 암호 알고리즘과 운영 모드의 성능을 비교했지만, 몇 가지 한계점이 존재한다. 첫째, 성능 측정은 더미 클라이언트를 사용하여 이루어졌으며, 실제 게임 클라이언트와는 일부 차이가 있을 수 있다. 다만, 더미 클라이언트와 유니티 클라이언트 간의 성능 차이가 거의 없음을 확인했으므로, 실제 환경에서도 유사한 결과가 나올 것으로 예상된다. 하지만, 게임 서버의 부하, 네트워크 상태, 다중 사용자 환경 등 더 복잡한 상황에 대한 추가적인 검증이 필요

할 것이다.

둘째, 본 연구에서는 패킷 크기에 따른 성능 변화를 고려하지 않았다. 일반적으로 패킷 크기가 커질수록 암호화 및 복호화 시간이 증가할 수 있으며, 이는 게임에서 다른 데이터의 크기나 유형에 따라 성능에 영향을 미칠 수 있다. 따라서, 향후 연구에서는 다양한 패킷 크기나 데이터 유형에 따른 성능 분석이 필요하다.

마지막으로, 본 연구에서는 보안성 분석이 포함되지 않았다. 본 연구는 성능에 중점을 두었기 때문에 각 블록 암호 알고리즘의 보안 강도에 대한 분석은 다루지 않았다. 암호화 알고리즘을 선택할 때는 성능뿐만 아니라 보안성도 중요한 고려 요소이므로, 각 암호화 알고리즘의 보안성 평가와 성능 간의 균형을 맞추는 추가 연구가 이루어져야 할 것이다.

7. 결론 및 향후 연구 방향

본 연구는 유니티 멀티플레이어 환경에서 안전한 통신을 위해 실제 게임 환경을 구축하고, 다양한 암호 알고리즘을 사용하였으며, 블록 암호 알고리즘과 운영모드의 성능을 비교 분석하였다. 이를 통해 각 암호화 방식의 성능 특성을 파악할 수 있었으며, 게임 개발자와 네트워크 프로토콜 설계자들이 성능과 보안 요구 사항에 맞는 암호화 방식을 선택하는 데 유용한 참고 자료를 제공하였다.

연구 결과, 대부분의 블록 암호 알고리즘이 실제 MMORPG 환경에서 레이턴시에 거의 영향을 미치지 않음을 확인했으며, 이는 어떤 암호 알고리즘을 적용하더라도 유저의 플레이 경험에 부정적인 영향을 주지 않음을 시사한다. 따라서, 본 연구에서 제안한 암호화 방식은 레이턴시가 중요한 게임에서도 충분히 적용 가능하다.

추가적으로, 본 연구에서는 게임 환경에서 보안을 강화하기 위해 블록 암호 알고리즘 뿐만 아니라 인증 및 키 교환 알고리즘도 성공적으로 구현하고 적용하였다. 시뮬레이션 및 실제 환경에서의 테스트 결과, 이러한 보안 기능들이 서버 - 클라이언트 간 통신에 성공적으로 적용되었으며, 이를 통해 게임 환경의 보안을 실질적으로 강화할 수 있음을 확인하였다.

향후 연구에서는 다음과 같은 부분에 대한 추가적인 연구가 필요하다.

- 1) 다중 사용자 테스트 : 실제 게임 환경에서 다수의 사용자가 참여하는 상황에서 암호화 성능을 분석하여, 다양한 네트워크 조건에서의 성능 변화를 평가할 필요가 있다.
- 2) 보안성 평가 : 암호화 알고리즘이 게임 환경에서 제공하는 보안 수준을 심도 있게 분석하고, 잠재적인 공격에 대한 대응 능력을 평가해야 한다.
- 3) 클라이언트 보안 강화 : EndPoint 인 클라이언트의 보안 또한 중요한 만큼, 본 연구에서 제안한 암호화 방식이 클라이언트 보안에도 적용될 수 있는지 추가적으로 연구할 필요가 있다.

이러한 후속 연구를 통해 본 연구 결과물을 더욱 발전시켜, 게임뿐만 아니라 다양한 네트워크 환경에서 안전한 통신을 보장할 수 있는 실질적인 보안 솔루션으로 확장할 수 있을 것이다.

8. 멘토링 결과 반영 내용

- 1) 공유 비밀을 그대로 대칭 키로 사용하는 경우는 거의 없음을 확인, 키 유도 함수를 통해 공유 비밀로부터 대칭 키를 유도하도록 변경함.
- 2) GCM-64는 표준 구현이 없고 안전성이 검증되지 않아 연구 주제를 벗어난다는 것을 확인, 구현에서 해당 내용을 제외함.
- 3) 암호 알고리즘의 구현 방식의 차이에 따른 성능 차이를 어느 정도 해소하기 위해 외부 라이브러리 사용을 최소화하고 모든 암호 알고리즘을 직접 구현함.

9. 참고 문헌

- [1] D. A. McGrew and J. Viega, "The Galois/Counter Mode of Operation (GCM)," NIST Technical Report, pp. 1-25, Jan. 2004.

[2] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Aug. 2018. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8446.txt>

[3] J. Thakkar, "TLS 1.3 Handshake: Taking a Closer Look," The SSL Store Blog, [Online]. Available: <https://www.thesslstore.com/blog/tls-1-3-handshake-tls-1-2/>

[4] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, Nov. 2001. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>

[5] "AES key schedule," Wikipedia, [Online]. Available: https://en.wikipedia.org/wiki/AES_key_schedule

[6] 한국인터넷진흥원(KISA), "민관겸용 블록 암호 알고리즘 ARIA 알고리즘 명세서," 2016. [Online]. Available: <https://www.kisa.or.kr/>

[7] 한국과학기술원(KAIST), "HIGHT 블록 암호 알고리즘 사양 및 세부 명세서," 2006. [Online]. Available: <https://www.kisa.or.kr/>

- [8] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, "Simon and Speck: Block Ciphers for the Internet of Things," National Security Agency, 9 July 2015. [Online]. Available: <https://eprint.iacr.org/2015/585.pdf>
- [9] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, "The Simon and Speck Families of Lightweight Block Ciphers," National Security Agency, 19 June 2013. [Online]. Available: <https://eprint.iacr.org/2013/404.pdf>
- [10] T. Suzaki, K. Minematsu, S. Morioka, and E. Kobayashi, "TWINE: A Lightweight, Versatile Block Cipher," NEC Corporation, 2012. [Online]. Available: <https://eprint.iacr.org/2012/677.pdf>
- [11] W. I. El Sobky, A. R. Mahmoud, A. S. Mohra, and T. El-Garf, "Enhancing Hierocrypt-3 Performance by Modifying Its S-Box and Modes of Operations," Proc. of the 2018 International Conference on Cryptology and Information Security in the Balkans, pp. 1-10, 2018.
- [12] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf>
- [13] "갈루아/카운터 모드," Wikipedia, [Online]. Available: https://ko.wikipedia.org/wiki/%EA%B0%88%EB%A3%A8%EC%99%80/%EC%B9%B4%EC%9A%B4%ED%84%B0_%EB%AA%A8%EB%93%9C (downloaded 2024, Oct. 18)
- [14] National Institute of Standards and Technology, "Cryptographic Algorithm Validation Program (CAVP)," [Online]. Available: <https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program> (downloaded 2024, Oct. 18)
- [15] 한국인터넷진흥원(KISA), "검증대상 암호알고리즘," [Online]. Available: <https://seed.kisa.or.kr/kisa/kcmvp/EgovVerification.do>
- [16] National Institute of Standards and Technology, "Digital Signature Standard (DSS)," FIPS PUB 186-4, July 2013. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-4.pdf>
- [17] L. Chen, S. Tomko, and R. Peralta, "Recommendations for Discrete Logarithm-Based Cryptography: Elliptic Curve Domain Parameters," NIST Special Publication 800-186, March 2023. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-186.pdf>

-
- [18] E. Barker, L. Chen, A. Roginsky, M. Smid, and E. Vassilev, "Recommendation for Pair-Wise Key Establishment Schemes Using Discrete Logarithm Cryptography," NIST Special Publication 800-56A Revision 3, April 2018. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-56Ar3.pdf>
- [19] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, May 2010. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5869.txt>
- [20] E. Barker, Q. Dang, and V. Souppaya, "Recommendation for Key-Derivation Methods in Key-Establishment Schemes," NIST Special Publication 800-56C Revision 2, National Institute of Standards and Technology, October 2020. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-56Cr2.pdf>
- [21] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, February 1997. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc2104.txt>
- [22] A. Jones, Network Programming For Microsoft Windows, 2nd Ed., pp. 232-235, Microsoft Press, Redmond, WA, 2002.